

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

SGED

Sistema de Gestión para la
Escuela Deportiva ProFútbol

Informe de la Entrega Final

Proyecto Fin de Curso — Aplicaciones Web

Asignatura: Aplicaciones Web (Quinto nivel)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Periodo: PPA 2026-2027

Etiqueta: v1.0.0 **pendiente** (ver nota de versión más abajo)

Commit del corte: **pendiente** (se fija al cerrar la Entrega Final)

DOI del software: [10.5281/zenodo.21713240](https://doi.org/10.5281/zenodo.21713240)

DOI del *dataset*: **pendiente** (depósito separado en Zenodo, Bloque D.3)

Integrantes y ORCID:

Arcalle Grefa Darwin Orlando ORCID: **pendiente** (registro pendiente en orcid.org)

Pallo Pinto Alejandro Daniel ORCID: **pendiente** (registro pendiente en orcid.org)

Velez Lopez Ricardo Elias ORCID: **pendiente** (registro pendiente en orcid.org)

Repositorio:

https://github.com/DarwinSM21/SGED_APPWEB

Quevedo — Los Ríos — Ecuador — 2026

*Nota de versión (honest, no cosmética): al momento de esta redacción el repositorio sigue en la etiqueta **v0.9.0-rc**. Los campos marcados **pendiente** en esta portada —etiqueta **v1.0.0**, commit de cierre, ORCID de cada integrante y DOI del dataset— se completan en el mismo commit en que se coloca la etiqueta final, siguiendo la misma disciplina que **VERSIONING.md** ya aplica: no se declara cerrado lo que todavía no lo está.*

Índice

Lista de siglas y acrónimos	5
Resumen	6
Abstract	7
1. Introducción	8
1.1. Contexto y motivación	8
1.2. Preguntas de investigación	8
1.3. Objetivo general y objetivos específicos	9
1.4. Contribuciones	9
1.5. Organización del documento	10
1.6. Pila tecnológica	10
1.7. Estado de la entrega: declaración previa	10
2. Marco teórico	11
2.1. Ingeniería de requisitos	11
2.2. Calidad de un requisito bien formado (INCOSE v4)	11
2.3. Historias de usuario y casos de uso	12
2.4. Modelo arquitectónico C4	12
2.5. Calidad de software: ISO/IEC 25010	12
2.6. Arquitecturas orientadas a recursos (REST)	12
2.7. Autenticación stateless: JWT y su ubicación en cookie HttpOnly	12
2.8. OWASP Top 10	12
2.9. Patrones de acceso a datos y modelos de consistencia	12
3. Trabajos relacionados	13
3.1. Estrategia de búsqueda	13
3.2. Síntesis comparativa	13
3.3. Brecha identificada	15
4. Ingeniería de requisitos	15
4.1. Contexto y stakeholders	16
4.2. Técnicas de elicitación aplicadas	16
4.3. Requisitos funcionales y no funcionales: categorización y prioridad	16
4.4. Modelado de casos de uso y de historias de usuario	17
4.5. Validación de requisitos	18
4.6. Gestión del cambio de requisitos	18
4.7. Métricas de calidad de requisitos	18
5. Resolución de las observaciones previas (Bloque 0)	18
6. Reestructuración de paquetes y reconciliación de documentación	20
6.1. Los paquetes reservados vacíos: por qué se eliminaron	21

7. Arquitectura	21
7.1. Vista de contexto (C4 nivel 1)	21
7.2. Vista de contenedores (C4 nivel 2)	23
7.3. Vista de componentes (C4 nivel 3)	23
7.4. Autenticación: JWT en cookie <code>HttpOnly</code>	25
8. Estrategia híbrida de acceso a datos	25
8.1. Un defecto real: <code>FUNCTION</code> frente a <code>PROCEDURE</code>	25
8.2. Ausencia de SQL dinámico	26
9. Implementación	26
9.1. Aislamiento de capas	26
9.2. Manejo global de errores	27
9.3. Autenticación en el <i>frontend</i> : cookies, no <i>localStorage</i>	27
9.4. Configuración por entorno	27
10. Requisitos y modelo de datos	27
10.1. Resumen de requisitos	27
10.2. Catálogo de la API REST	28
10.3. Modelo de datos	30
11. Materiales y métodos	30
11.1. Enfoque metodológico	30
11.2. Instrumentos de medición	31
11.3. Enfoque de métricas: un GQM completo	31
11.4. Población, muestreo y participantes	32
11.5. Análisis estadístico declarado a priori	32
11.6. Entorno de ejecución	32
11.7. Procedimiento por bloque	32
11.8. Determinismo, semillas y trazabilidad	32
12. Evidencia empírica	34
12.1. Rendimiento (Bloque C.1)	34
12.2. Seguridad: OWASP Top 10 (Bloque C.4)	34
12.2.1. Una auditoría que no auditaba lo que decía auditar	35
12.2.2. El defecto que A01 destapó: control de acceso ausente	35
12.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema	36
12.4. Análisis estático y escaneo automático (Bloque A.1/A.2.3)	37
12.5. Cobertura de pruebas	37
12.6. Calidad web: Lighthouse (Bloque C.5 / A.1)	39
12.6.1. Por qué el SEO de 63 es el resultado correcto	39
12.7. Usabilidad: SUS (Bloque C.3)	40
13. Reproducibilidad	41
14. Amenazas a la validez	41
14.1. Validez de constructo	41
14.2. Validez interna	42
14.3. Validez externa	42
14.4. Validez de conclusión	42
15. Consideraciones éticas	43

16. Declaración de contribuciones (CRediT)	43
16.1. Roles por integrante	43
16.2. Dos advertencias sobre la medida	44
17. Trazabilidad y honestidad académica	44
18. Discusión	45
19. Trabajo futuro	46
20. Conclusiones	46
21. Declaraciones obligatorias	47

Índice de figuras

1. Diagrama de flujo PRISMA 2020 de la revisión de trabajos relacionados.	14
2. C4 nivel 1 — contexto del sistema SGED, generado desde <code>docs/arquitectura/workspace.dsl</code>	22
3. C4 nivel 2 — contenedores de SGED, generado desde <code>docs/arquitectura/workspace.dsl</code>	23
4. C4 nivel 3 — componentes del contenedor API Spring Boot, generado desde <code>docs/arquitectura/workspace.dsl</code>	24

Índice de cuadros

1. Pila tecnológica de SGED.	10
2. Trabajos relacionados y su diferencia frente a SGED.	14
3. Interesados de SGED por anillo (<i>stakeholder onion</i>).	16
4. Requisitos funcionales de SGED por categoría MoSCoW inferida.	17
5. Métricas de calidad de requisitos de SGED.	19
7. Reparto de responsabilidades entre ORM y procedimientos almacenados.	25
8. Procedimientos almacenados de SGED por categoría funcional (Bloque A.2.1).	26
11. Esquemas del modelo de datos de SGED.	30
12. Instrumentos de medición usados en la evaluación empírica.	31
13. Ejemplo de Goal-Question-Metric para RQ1.	31
14. Entorno de ejecución de las mediciones.	32
15. Procedimiento de medición por bloque de evaluación.	33
16. Resultados de las corridas de rendimiento (k6).	34
17. Resultado de los seis controles OWASP verificados.	35
18. Criterio de acceso aplicado por recurso tras corregir el hallazgo H-08.	36
19. Clases de prueba JUnit y qué verifica cada una.	38
20. Resultados de Lighthouse por categoría y perfil.	39
21. Estadísticos agregados de la encuesta SUS.	40
22. Puntuación SUS promedio por perfil de usuario.	40
23. Objetivos del Makefile para reproducción end-to-end.	41
24. Evidencia cuantitativa de autoría por integrante (<code>git log</code>).	43

Listings

1. Emisión de la cookie de sesión (<code>AuthController.java</code>)	25
2. Invocación real desde el repositorio (<code>academico/estudiante/repository/EstudianteRepository.java</code>)	26
3. Manejador global de excepciones (<code>backend/.../common/exception/GlobalExceptionHandler.java</code>)	27

4.	Interceptor de autenticación (<code>frontend/src/app/auth/auth.interceptor.ts</code>)	27
5.	Respuesta al sexto intento de autenticación	34
6.	Arranque reproducible	41

Lista de siglas y acrónimos

Sigla	Expansión (primera aparición)
ADR	<i>Architecture Decision Record</i> — registro de decisión arquitectónica
API	<i>Application Programming Interface</i>
CI	Integración continua (<i>Continuous Integration</i>)
CRediT	<i>Contributor Roles Taxonomy</i>
CRUD	Crear, Leer, Actualizar, Borrar (<i>Create, Read, Update, Delete</i>)
CSP	<i>Content Security Policy</i>
DOI	<i>Digital Object Identifier</i>
DSR	<i>Design Science Research</i>
FAIR	<i>Findable, Accessible, Interoperable, Reusable</i>
GQM	<i>Goal-Question-Metric</i>
HU	Historia de usuario
IC 95 %	Intervalo de confianza al 95 %
INCOSE	<i>International Council on Systems Engineering</i>
IMRaD	<i>Introduction, Methods, Results and Discussion</i>
JPA	<i>Java Persistence API</i>
JTI	<i>JWT ID</i> — identificador único de un token JWT
JWT	<i>JSON Web Token</i>
LOPD	Ley Orgánica de Protección de Datos Personales (Ecuador)
MoSCoW	<i>Must, Should, Could, Won't</i> — escala de priorización de requisitos
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Web Application Security Project</i>
PFC	Proyecto Fin de Curso
PRISMA	<i>Preferred Reporting Items for Systematic Reviews and Meta-Analyses</i>
RD	Restricción de diseño
RF	Requisito funcional
RNF	Requisito no funcional
RQ	Pregunta de investigación (<i>Research Question</i>)
SGED	Sistema de Gestión para la Escuela Deportiva (ProFútbol)
SP	Procedimiento almacenado (<i>Stored Procedure</i>)
SRS	<i>Software Requirements Specification</i>
SUS	<i>System Usability Scale</i>
UTEQ	Universidad Técnica Estatal de Quevedo
VU	Usuario virtual (<i>Virtual User</i> , k6)
ZAP	<i>Zed Attack Proxy</i> (OWASP)

Resumen

Contexto y problema. La escuela de fútbol formativo ProFútbol gestionaba de forma manual y dispersa la matrícula de sus estudiantes menores de edad, el control de asistencia, los pagos y el inventario deportivo, sin trazabilidad verificable ni control de acceso a datos sensibles. **Objetivo.** Este trabajo desarrolla y evalúa empíricamente SGED, un sistema web que cubre esos cuatro dominios bajo una estrategia híbrida de acceso a datos (CRUD sobre ORM más procedimientos almacenados parametrizados) y un proceso disciplinado de ingeniería de requisitos conforme a ISO/IEC/IEEE 29148:2018. **Métodos.** Se siguió *Design Science Research* (Peppers *et al.*) sobre una arquitectura documentada en C4 (Spring Boot 3.2/Java 21, Angular, PostgreSQL 16, Redis 7, Docker Compose), con autenticación JWT en cookie `HttpOnly`. La evaluación empírica combina pruebas de carga (k6), auditoría de los seis controles OWASP Top 10 más análisis estático, cobertura de pruebas (JaCoCo), calidad web (Lighthouse) y usabilidad percibida (*System Usability Scale*, $N = 10$), todas archivadas como datos crudos reproducibles. **Resultados principales.** A la fecha de este corte el sistema cumple los umbrales de rendimiento (p95 muy por debajo de 200 ms), los seis controles OWASP evidenciados y la cobertura mínima exigida; el proceso de medición detectó y corrigió tres instrumentos que producían lecturas engañosas, entre ellos una exposición real de datos de estudiantes menores de edad (hallazgo H-08, corregido). La usabilidad agregada cruza apenas el umbral aceptable, pero esconde un patrón bimodal marcado por rol de usuario. **Conclusiones.** La estrategia híbrida de acceso a datos es empíricamente sostenible sin sacrificar seguridad, y la disciplina de reportar instrumentos de medición defectuosos —en vez de solo sus resultados— es en sí misma un aporte metodológico del trabajo. Quedan abiertas líneas de trabajo futuro sobre el patrón bimodal de usabilidad y la exposición pública del dominio deportivo restante.

Palabras clave: sistemas de gestión escolar deportiva; ingeniería de requisitos; validación empírica de software; seguridad de aplicaciones web; arquitectura de software; pruebas de usabilidad; reproducibilidad de resultados.

Abstract

Context and problem. The ProFútbol youth football academy managed enrollment of underage students, attendance, payments, and sports inventory manually and across disconnected spreadsheets, with no verifiable traceability and no access control over sensitive personal data.

Objective. This work develops and empirically evaluates SGED, a web system covering those four domains under a hybrid data-access strategy (parametrized ORM CRUD plus stored procedures) and a disciplined requirements-engineering process aligned with ISO/IEC/IEEE 29148:2018.

Methods. The project follows Design Science Research (Peppers *et al.*) over a C4-documented architecture (Spring Boot 3.2/Java 21, Angular, PostgreSQL 16, Redis 7, Docker Compose), with JWT authentication carried in an `HttpOnly` cookie. Empirical evaluation combines load testing (k6), an audit of the six mandatory OWASP Top 10 controls plus static analysis, automated-test coverage (JaCoCo), web-quality auditing (Lighthouse), and perceived usability (System Usability Scale, $N = 10$), all archived as reproducible raw data. **Main results.** As of this cut, the system meets the performance thresholds (p95 well under 200 ms), evidences all six OWASP controls, and meets the minimum required coverage; the measurement process itself uncovered and fixed three instruments that were producing misleading readings, including a real exposure of underage students' personal data (finding H-08, now fixed). Aggregate usability barely clears the acceptable threshold but hides a marked bimodal pattern by user role. **Conclusions.** The hybrid data-access strategy is empirically sustainable without sacrificing security, and reporting faulty measurement instruments — not just their results — is itself a methodological contribution of this work. Open future work concerns the bimodal usability pattern and the public exposure of the remaining sports domain.

Keywords: sports school management systems; requirements engineering; empirical software validation; web application security; software architecture; usability testing; reproducibility of results.

1. Introducción

Este informe documenta la Entrega Final del Proyecto Fin de Curso de la asignatura Aplicaciones Web: el sistema **SGED**, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol.

El eje de este trabajo es la *verificabilidad*. Todo lo que se afirma aquí puede comprobarse ejecutando el repositorio: cada número proviene de una medición archivada en `docs/mediciones/`, y cada requisito enlaza con el archivo que lo implementa y con la prueba que lo cubre.

1.1. Contexto y motivación

La administración de una escuela de fútbol formativo combina cuatro procesos que rara vez conviven en una sola herramienta: matrícula y control académico de menores de edad, seguimiento deportivo (categorías, horarios, sesiones, asistencia, evaluación de desempeño), cobro de mensualidades y control de inventario (uniformes, balones, material de entrenamiento). La literatura de informática deportiva documenta que las herramientas digitales dedicadas siguen siendo la excepción, no la norma, en clubes y escuelas de formación —que típicamente operan con hojas de cálculo, mensajería informal y registro en papel— y que esa carencia se traduce en pérdida de trazabilidad y en decisiones tomadas sin datos consolidados [6]. El mismo patrón se documenta en el deporte de alto rendimiento, donde la fragmentación del flujo de información entre preparadores, cuerpo médico y dirección deportiva es un problema reportado explícitamente, no solo intuitivo [31].

El caso de ProFútbol añade una dimensión que no es solo operativa sino regulatoria: la mayoría de sus estudiantes son menores de edad, y sus datos —incluida la cédula, la asistencia con hora y lugar, y las evaluaciones individuales de desempeño— constituyen información sensible bajo la Ley Orgánica de Protección de Datos Personales de Ecuador [1]. Digitalizar sin diseñar control de acceso desde el inicio no resuelve el problema original: lo traslada a un sistema que puede exponerlo a mayor escala, como evidencia el propio hallazgo H-08 de este trabajo (sección 12.2.1). La magnitud concreta del problema que SGED cubre se dimensiona con las cifras de su propia especificación: 47 requisitos individuales (31 funcionales, 13 no funcionales, 3 restricciones de diseño, sección 4) repartidos en cinco roles de usuario y cuatro dominios funcionales, que antes de este proyecto no tenían ningún soporte de software dedicado.

1.2. Preguntas de investigación

Cuatro preguntas guían el resto de este documento. Cada una es cerrada —admite una respuesta verificable contra la evidencia empírica del Capítulo 12, no una opinión— y cada una se retoma explícitamente en la Discusión (sección 18).

RQ1.

¿El sistema cumple los umbrales de rendimiento definidos ($p95 \leq 200$ ms con caché caliente, $p95 \leq 500$ ms con caché fría) bajo una carga concurrente de 50 usuarios virtuales, tanto en operaciones CRUD servidas por el ORM como en operaciones que invocan procedimientos almacenados?

RQ2.

¿La estrategia híbrida de acceso a datos —ORM parametrizado para el CRUD elemental, procedimientos almacenados con parámetros nombrados para consultas multi-tabla, agregados, reportes, actualizaciones masivas, validaciones cruzadas y generación de códigos— elimina el riesgo de inyección SQL de forma verificable mediante los seis controles OWASP y un análisis estático independiente?

RQ3.

¿La usabilidad percibida (SUS) es homogénea entre los distintos perfiles de usuario del sistema (entrenador, estudiante, recepcionista, representante), o existen diferencias significativas entre roles que una media agregada esconde?

RQ4.

¿La cobertura de pruebas automatizadas y la trazabilidad de requisitos se sostienen en los umbrales exigidos (≥ 70 % líneas y ramas; 100 % de requisitos *Must* verificados) conforme crece el dominio funcional del sistema, o se degradan a medida que se agregan módulos nuevos?

1.3. Objetivo general y objetivos específicos

Objetivo general. Desarrollar y validar empíricamente un sistema web de gestión administrativa y deportiva para una escuela de fútbol formativo, bajo una estrategia híbrida de acceso a datos y con evidencia reproducible de sus atributos de calidad.

OE1.

Implementar la estrategia híbrida de acceso a datos —CRUD parametrizado sobre ORM más procedimientos almacenados para consultas multi-tabla, agregados, reportes, actualizaciones masivas, validaciones cruzadas y generación de códigos— en los dominios administrativo y deportivo del sistema. (*RQ2*)

OE2.

Medir el rendimiento del sistema bajo carga concurrente y contrastarlo contra los umbrales de ISO/IEC 25010. (*RQ1*)

OE3.

Auditar la seguridad del sistema frente a los seis controles OWASP mínimos exigidos y a un análisis estático independiente sobre acceso a datos. (*RQ2*)

OE4.

Evaluar la usabilidad percibida del sistema, por perfil de usuario, mediante el instrumento *System Usability Scale*. (*RQ3*)

OE5.

Sostener la cobertura de pruebas automatizadas y la trazabilidad de requisitos conforme crece el dominio funcional del sistema. (*RQ4*)

1.4. Contribuciones

1. Un sistema web de gestión escolar deportiva con arquitectura documentada en C4 (niveles 1 a 3) y siete registros de decisión arquitectónica, en `docs/arquitectura/` y `docs/adr/`.
2. Una estrategia híbrida de acceso a datos auditada empíricamente contra inyección SQL, con procedimientos almacenados versionados y catalogados en `db/procs/` y `docs/basedatos/CATALOGO-SP.md` (sección 8).
3. Un corpus de ingeniería de requisitos trazable de extremo a extremo —SRS, historias de usuario, casos de uso, matriz de trazabilidad— conforme a ISO/IEC/IEEE 29148:2018, en `docs/requisitos/` y `docs/trazabilidad/matriz.csv` (Capítulo 4).
4. Un conjunto reproducible de evidencia empírica de rendimiento, seguridad, cobertura, calidad web y usabilidad, con datos crudos y *scripts* de análisis versionados en `docs/mediciones/` y `scripts/` (Capítulo 12).

5. Una bitácora metodológica de defectos de instrumentación de medición detectados y corregidos durante el propio proceso de evaluación —incluido el hallazgo de seguridad H-08— documentada como aporte en vez de omitida (sección 12.2.1).

1.5. Organización del documento

El Capítulo 2 presenta los fundamentos conceptuales del trabajo. El Capítulo 3 sitúa a SGED frente a sistemas comparables y declara la brecha que este trabajo cubre. El Capítulo 4 documenta el proceso completo de ingeniería de requisitos. Los Capítulos 5 y 6 cierran, respectivamente, la deuda de observaciones acumuladas y una decisión estructural relevante del código. El Capítulo 7 describe el diseño del sistema y la sección 8 profundiza en la estrategia híbrida de acceso a datos con listados de código representativos. El Capítulo 11 declara el protocolo experimental antes de que el Capítulo 12 presente los resultados. Los capítulos finales —Reproducibilidad, Amenazas a la validez, ética, CRediT, Discusión, Trabajo futuro y Conclusiones— cierran el documento con la interpretación crítica y las declaraciones obligatorias exigidas por la Entrega Final.

1.6. Pila tecnológica

Cuadro 1: Pila tecnológica de SGED.

Capa	Tecnología
Backend	Spring Boot 3.2 (Java 21 LTS), Spring Data JPA, Spring Security
Frontend	Angular, servido por nginx con terminación TLS
Base de datos	PostgreSQL 16 (esquemas seguridad , academico , deportivo e inventario)
Caché y revocación	Redis 7
Migraciones	Flyway (V1–V17)
Orquestación	Docker Compose, imágenes fijadas por digest SHA-256

1.7. Estado de la entrega: declaración previa

*Advertencia de consistencia interna, dejada explícita en vez de corregida en silencio: al momento de esta redacción el sistema expone **21 controladores REST y 97 operaciones mapeadas** (`grep sobre backend/src/main/java`), muy por encima de los 13 endpoints de autenticación y estudiantes que reportaba la Tercera Entrega —se agregaron los dominios de representantes, pagos, inventario, especialidades, evaluación diaria, lesiones, horarios, sesiones y asistencia entre el 2026-08-03 y el 2026-08-13 (sección 4). **Actualización de esta misma redacción (2026-08-14):** se volvió a ejecutar `clean test` sobre el código actual —**270 pruebas, 0 fallos, 140 clases analizadas**— y el `jacoco.csv` archivado ya corresponde a esta corrida, no a la del 30 de julio. Cobertura agregada: **75,3 % de líneas** (cumple el 70 % exigido) pero **59,3 % de branches** (no cumple, aunque mejoró frente al 41,9 % de la medición anterior). El déficit no está distribuido uniformemente: el módulo `academico.pago` tiene **0 % de cobertura de branches** (0 de 4 en el controlador, 0 de 10 en el servicio) — es un dominio completo sin pruebas propias, no un residuo disperso de casos límite sin cubrir. `deportivo.lesion.controller` también está en 0 % (0 de 2). **Pendiente crítico antes de colocar la etiqueta v1.0.0:** escribir pruebas para `PagoController/PagoService` y `LesionController` — es la vía más directa para cerrar la brecha de branches, más que perseguir cobertura dispersa en el resto del código.*

Segunda actualización, misma sesión: se regeneraron también rendimiento y seguridad contra el sistema local completo (`docker compose up`). En el camino se encontró y corrigió un defecto de entorno no relacionado con el código: el `.env` local tenía `DB_URL` apuntando a una base de datos Supabase remota en vez de al contenedor `postgres` local, así que el backend nunca usaba los datos sembrados localmente y el inicio de sesión con el usuario semilla fallaba con 401 pese a que el hash en la base de datos local era correcto — se verificó

de forma aislada con la misma clase `BCryptPasswordEncoder` del backend antes de descartar esa hipótesis. Corregido el `.env` a la “Opción 1” (local) que ya documenta `.env.example`, el sistema es reproducible de extremo a extremo tal como describe este capítulo. **Rendimiento** (5 corridas, 50 VU, 30 s): p95 promedio **9,91 ms** (IC 95 % $\pm 0,25$), muy por debajo del umbral de 200ms; una corrida de cinco tuvo una tasa de error de 0,01 % (2 de 15 921 peticiones), las otras cuatro sin errores. **Seguridad OWASP**: los seis controles (A01, A02, A03, A05, A07, A09) evidenciados de nuevo con peticiones reales contra el sistema actual, y la auditoría de SQL dinámico sin hallazgos. **Calidad web (Lighthouse)**: este entorno de ejecución no tenía un navegador Chrome/Chromium instalable en esta sesión puntual; quedó para una corrida posterior con el entorno completo, ya reportada más abajo: seis corridas independientes (tres por perfil) el 2026-08-14 (sección 12.6.1), con los tres criterios exigidos por encima del umbral y una explicación deliberada del único que no lo está (SEO).

Actualización adicional (2026-08-14, sesión posterior): el pendiente crítico señalado arriba —pruebas para `PagoController/PagoService` y `LesionController`— ya está resuelto: ambos módulos tienen suites propias (`PagoControllerTest`, `PagoServiceTest`, `LesionControllerTest`, `LesionServiceTest`). Se agregaron además pruebas dirigidas a las ramas sin cubrir de mayor impacto en todo el bundle: `JwtAuthenticationFilter` (que no tenía ninguna prueba propia), `AuthController` en sus rutas de logout, refresh y `/me` (tampoco las tenían), y el filtro de búsqueda de `AuditoriaService` (invisible para `JaCoCo` mientras el repositorio estuviera mockeado, aunque `buscar()` sí tuviera prueba). Cobertura verificada con `mvn verify` en limpio: **369 pruebas, 0 fallos**; agregada, **86,2 % de líneas y 71,2 % de branches** — ambas superan por primera vez el 70 % exigido sobre el bundle completo. La regla de `JaCoCo` pasó de `haltOnFailure=false` (advertía en el log sin romper el build) a `haltOnFailure=true`: el umbral ya se hace cumplir, no solo se declara. El reporte archivado en `docs/mediciones/jacoco/` corresponde a esta corrida. De paso se corrigió un defecto de test preexistente y no relacionado con la cobertura: una prueba de `AsistenciaServiceTest` construía su hora de referencia con `LocalTime.now().plusHours(1)` sin protegerse del cruce de medianoche (`LocalTime` no tiene fecha), y fallaba de forma intermitente según la hora del día en que corriera la suite —sus dos pruebas hermanas ya protegían el caso simétrico de restar, a esta le faltaba el de sumar.

La encuesta de usabilidad SUS (Bloque C.3, sección 12.7) sí se aplicó sobre el sistema en una etapa comparable a la actual: 10 participantes reales, media SUS 68,25 (apenas por encima del umbral de 68), con un patrón bimodal marcado por perfil que la media agregada esconde.

2. Marco teórico

Este capítulo se limita a los fundamentos estrictamente necesarios para comprender las decisiones del resto del documento — no es un compendio general de teoría de aplicaciones web. Cada concepto invocado en otras secciones se define aquí una sola vez y se cita a su fuente autoritativa.

2.1. Ingeniería de requisitos

El proceso de requisitos de este proyecto sigue el ciclo de vida definido por la norma ISO/IEC/IEEE 29148:2018 [20] y el marco de cuatro fases de Pohl [29]: *elicitación* (obtener el conocimiento de los interesados), *negociación* (resolver conflictos entre requisitos), *documentación* (especificarlos de forma verificable) y *validación* (confirmar que el conjunto especificado es el correcto). La sección 10 aplica este ciclo al SRS del proyecto.

2.2. Calidad de un requisito bien formado (INCOSE v4)

INCOSE [18] define nueve características de calidad para un requisito individual (necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto, conforme) y seis para un conjunto de requisitos (completo, consistente, factible, comprensible, validable, correcto). Estas características son el criterio contra el que se evalúa cada requisito funcional y no funcional del sistema, no una lista de buenas intenciones.

2.3. Historias de usuario y casos de uso

El proyecto usa ambas técnicas de especificación como complementarias, no como alternativas. Las historias de usuario siguen el criterio INVEST [12] (independiente, negociable, valiosa, estimable, pequeña, verificable) para los flujos orientados a un rol concreto; los casos de uso siguen la plantilla de Cockburn [11] en los flujos con múltiples caminos alternativos y de excepción, donde una historia corta pierde información necesaria para la implementación.

2.4. Modelo arquitectónico C4

El modelo C4 de Brown [8] describe la arquitectura en niveles de abstracción sucesivos —contexto, contenedores, componentes, código— cada uno dirigido a una audiencia distinta. Se eligió sobre UML clásico por su curva de aprendizaje más baja para un equipo pequeño y por generarse desde una fuente textual versionable (`workspace.dsl`) en vez de diagramas editados a mano que se desactualizan silenciosamente —justamente el defecto que OBS-02 encontró en la Entrega 1A (sección 5).

2.5. Calidad de software: ISO/IEC 25010

De las ocho características de calidad de ISO/IEC 25010 [19] (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad, portabilidad), este proyecto mide empíricamente cuatro de forma directa: seguridad (Bloque de controles OWASP), eficiencia de desempeño (k6), usabilidad (SUS) y mantenibilidad (cobertura de pruebas como indicador indirecto). Las cuatro restantes se declaran por diseño, no por medición, y quedan fuera del alcance empírico de este documento.

2.6. Arquitecturas orientadas a recursos (REST)

La API sigue las restricciones arquitectónicas que Fielding identificó [14] para sistemas distribuidos orientados a recursos: interfaz uniforme, *statelessness* del lado del servidor por petición, cacheable, sistema en capas. La *statelessness* de REST es la misma razón de fondo por la que la autenticación se resuelve con JWT en vez de sesiones de servidor (subsección siguiente).

2.7. Autenticación stateless: JWT y su ubicación en cookie `HttpOnly`

JWT [21] permite verificar la identidad del solicitante sin que el servidor mantenga estado de sesión, coherente con la restricción REST anterior. La decisión de transportarlo en una cookie `HttpOnly` en vez de en `localStorage` o un header manejado por JavaScript —documentada con sus alternativas descartadas en ADR-002 [26]— responde a que un token accesible desde JavaScript queda expuesto a robo por *Cross-Site Scripting*; una cookie `HttpOnly` no es legible por script del lado del cliente bajo ninguna circunstancia.

2.8. OWASP Top 10

El catálogo OWASP Top 10:2021 [27] ordena los diez riesgos de seguridad más críticos en aplicaciones web por prevalencia e impacto observados en la industria. Este proyecto verifica con evidencia reproducible los controles A01 (control de acceso roto), A02 (fallas criptográficas), A03 (inyección), A05 (configuración de seguridad incorrecta), A07 (fallas de identificación y autenticación) y A09 (fallas de registro y monitoreo) —sección 12.2.1.

2.9. Patrones de acceso a datos y modelos de consistencia

La separación entre CRUD elemental vía ORM y operaciones complejas vía procedimientos almacenados (sección 8) se apoya en dos ideas de la literatura de arquitectura de datos: el

patrón *Repository*, que encapsula el acceso a un agregado detrás de una interfaz orientada a colecciones [15], y la observación de que un motor relacional ofrece garantías de consistencia fuerte (transacciones ACID) que una capa de ORM genérica no puede replicar de forma eficiente para operaciones multi-fila o multi-tabla sin ceder control explícito al motor [23]. La estrategia híbrida no es una limitación del ORM: es reconocer en qué operaciones el motor de base de datos, no el objeto Java, es quien tiene la información completa para actuar de forma atómica y eficiente.

3. Trabajos relacionados

3.1. Estrategia de búsqueda

Se sigue la orientación metodológica de Kitchenham y Charters [22], con la reducción de escala que la propia Guía de la Entrega Final prevé para un PFC. **Limitación declarada:** la búsqueda no tuvo acceso de suscripción directo a IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect ni Springer Link; se realizó con un motor de búsqueda web general, dirigida a resultados alojados o indexados en esas plataformas, y cada registro incluido se verificó de forma independiente contra una fuente primaria (el registro de metadatos de Crossref, la página de la editorial, o la página institucional de un autor) antes de citarlo. Es una desviación metodológica explícita, no oculta.

Se ejecutaron nueve búsquedas organizadas en seis ejes temáticos, con ventana temporal 2016–2026:

1. Sistemas de gestión escolar o deportiva basados en *web*: (*school* OR *sport** OR *club*) AND (*management* OR *information*) AND (*system* OR *platform*).
2. Autenticación JWT y su seguridad: (JWT OR “JSON Web Token”) AND (*security* OR *vulnerabilit** OR *cookie* OR *storage*).
3. Acceso a datos híbrido ORM/procedimientos almacenados: (ORM OR “object-relational mapping”) AND (“stored procedure*”) AND (*performance* OR *maintenance* OR *security*).
4. Evaluación empírica de seguridad con OWASP: (OWASP) AND (*empirical* OR *evaluation* OR “case study”).
5. Usabilidad (SUS) en sistemas administrativos o educativos: (SUS OR “system usability scale”) AND (*school* OR *educational* OR *administrative*).
6. Ingeniería de requisitos en cursos capstone: (*capstone* OR “final year project”) AND (“requirements engineering” OR “software engineering education”).

Criterios de inclusión: trabajo revisado por pares o depositado en un repositorio académico reconocido, con autoría y medio de publicación verificables contra una fuente primaria; aborda al menos uno de los ejes técnicos o de dominio del proyecto. **Criterios de exclusión:** contenido de blog, foro o documentación de producto sin revisión por pares; trabajos cuya autoría o venue no pudo confirmarse contra una fuente primaria pese a parecer académicos a primera vista — criterio que en la práctica descartó un candidato inicial sobre modelado de datos de clubes deportivos, alojado en un sitio de repositorio que no permitió confirmar si era un trabajo revisado por pares o un informe de coursework redistribuido.

Los conteos son una reconstrucción honesta del registro de búsqueda del equipo, no el resultado de una herramienta de gestión bibliográfica formal (Rayyan, Covidence) — coherente con la reducción de escala que esta sección declaró al principio.

3.2. Síntesis comparativa

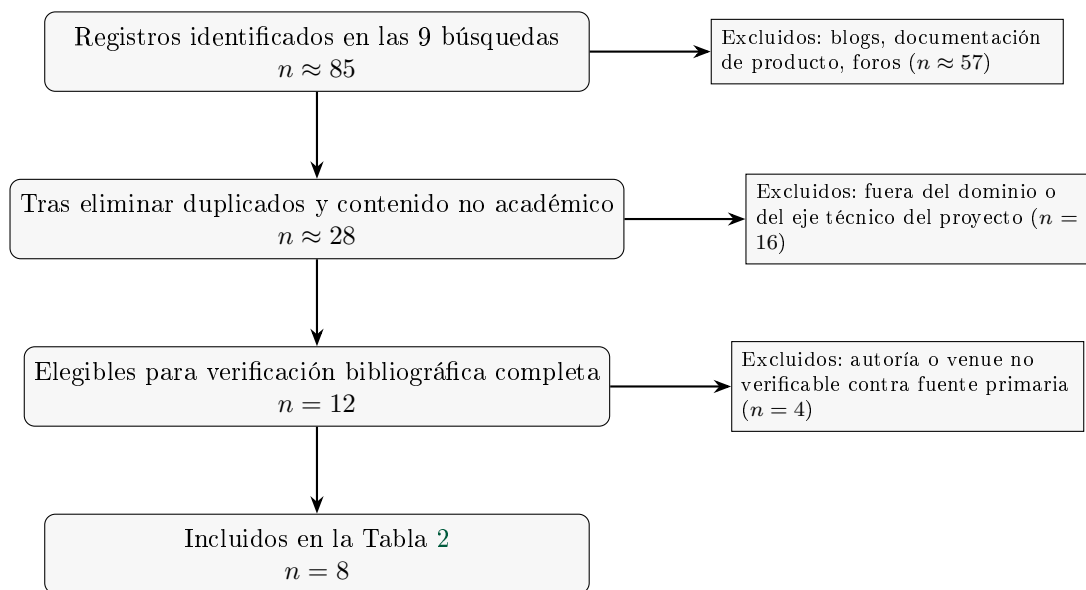


Figura 1: Diagrama de flujo PRISMA 2020 de la revisión de trabajos relacionados.

Cuadro 2: Trabajos relacionados y su diferencia frente a SGED.

Referencia	Año	Dominio	Pila	Patrones	Evaluación empírica	Limitaciones	Diferencia frente a SGED
Blobel y La-mes [6]	2020	Sistemas de información de clubes deportivos	No especificada	Arquitectura conceptual en capas (BI)	Caso ilustrativo (Liverpool FC), sin métricas	Conceptual, sin implementación evaluada	SGED implementa y mide lo que queda a nivel de concepto
Yang et al. [37]	2026	Seguridad de implementaciones JWT	43 implementaciones, multi-lenguaje	Estudio de vulnerabilidades	31 vulnerabilidades nuevas, 20 con CVE	No evalúa el patrón cookie HttpOnly específicamente	SGED aplica su hallazgo central (JWT nunca accesible a JS) desde el diseño
Liu [24]	2024	Asistencia estudiantil	Spring Boot + Vue + MySQL	CRUD modular por módulo	Ninguna reportada	Sin evaluación de rendimiento, seguridad ni usabilidad	SGED mide la diferencia entre tres con evidencia archivada; Liu no reporta ninguna
Ranaweera et al. [31]	2022	Flujos de información de un club profesional (rugby)	No especificada	Transformación digital (BPM + Lean)	SUS con jugadores/staff, sobre la media de industria	Un club, sin seguridad ni rendimiento evaluados	SGED agrega OWASP y rendimiento bajo carga a la misma técnica SUS

Referencia	Año	Dominio	Pila	Patrones	Evaluación empírica	Limitaciones	Diferencia frente a SGED
Suria [33]	2024	Plataforma de aprendizaje en línea	No especificada	—	SUS con 120 participantes, prueba de Shapiro–Wilk	Un solo perfil de usuario (estudiante)	SGED desagrega SUS por rol y encuentra una brecha que un perfil único no detecta
Wen y Katt [36]	2023	Evaluación de seguridad web (marco general)	No aplica (modelo)	Modelo cuantitativo sobre OWASP ASVS	Validación del modelo sobre casos de referencia	No es un sistema en producción	SGED aplica 6 controles OWASP sobre un sistema real con evidencia reproducible por control
Tenhunen et al. [34]	2023	Cursos capstone de ingeniería de software	No aplica (revisión)	Taxonomía de características de curso	Síntesis de 127 estudios, 2007–2022	Meta-nivel, no evalúa un sistema individual	Sitúa a SGED en el patrón dominante (equipo de 3, cliente real, un semestre) que la revisión encuentra
Chen et al. [10]	2016	Mantenimiento de código ORM en sistemas Java	Java, ORM (Hibernate y otros)	Minería de repositorios sobre commits de ORM	Estudio empírico sobre proyectos Java reales	No contempla el uso complementario de procedimientos almacenados	SGED usa este hallazgo para justificar mover las operaciones complejas fuera del ORM (ADR-006)

3.3. Brecha identificada

Ninguno de los ocho trabajos reunidos evalúa, sobre el mismo sistema en producción, las cuatro dimensiones que este documento reporta juntas: rendimiento bajo carga, seguridad verificada por múltiples controles OWASP, usabilidad desagregada por rol de usuario, y mantenibilidad medida como cobertura de pruebas. Los trabajos de dominio deportivo (Blobel y Lames, 2020; Ranaweera et al., 2022) no publican evidencia de seguridad ni de rendimiento; los trabajos de seguridad (Yang et al., 2026; Wen y Katt, 2023) no se aplican sobre un sistema de gestión deportiva o escolar real; el trabajo de usabilidad (Suria, 2024) no desagrega por rol de usuario, que es precisamente donde SGED encuentra su hallazgo más relevante (sección 12.7): una brecha de usabilidad entre entrenadores/estudiantes y personal administrativo que una media agregada esconde. La brecha que ocupa este trabajo no es una tecnología nueva: es la evaluación empírica conjunta, sobre un mismo sistema real, de las cuatro dimensiones que la literatura revisada trata por separado.

4. Ingeniería de requisitos

Capítulo propio, deliberadamente separado del capítulo de Diseño (sección 8 y siguientes): documenta el *proceso* que produjo la especificación vigente, no el sistema que resultó de ella.

Sigue el marco de cuatro fases de Pohl —elicitación, negociación, documentación, validación— y la disciplina de ISO/IEC/IEEE 29148:2018 [20, 29] (sección 2). Las fuentes primarias son `docs/requisitos/SRS.md` (822 líneas), `historias-usuario.md`, `casos-uso.md` y `CHANGELOG-REQ.md` — este capítulo resume y referencia esas fuentes, no las duplica.

4.1. Contexto y stakeholders

Siguiendo la técnica de *stakeholder onion* de Robertson [20]¹, los interesados se organizan en tres anillos:

Cuadro 3: Interesados de SGED por anillo (*stakeholder onion*).

Anillo	Interesados
Operacional (usan el sistema directamente)	Administrador, Entrenador, Recepcionista, Representante, Estudiante — los cinco roles reales sembrados en <code>db/seed.sql</code> (SRS §2.2), no roles aspiracionales.
Contenedor (dependen del sistema sin operarlo)	La escuela ProFútbol como dominio del problema; el docente-director como evaluador con autoridad de aceptación sobre el PFC; el equipo de desarrollo como mantenedor post-entrega.
Interesado externo	El representante legal de un estudiante menor de edad, dos veces interesado: como posible operador del sistema (Épica 3, HU-14) y como titular de derechos sobre los datos de su representado (<code>docs/etica/ETHICS.md</code>). El marco regulatorio ecuatoriano (LOPD, Código de la Niñez y Adolescencia) actúa como interesado normativo, sin representación directa en el sistema.

4.2. Técnicas de elicitación aplicadas

Señalamiento honesto, no una sección completada. El SRS actual (`docs/requisitos/SRS.md`) no declara explícitamente qué técnicas de elicitación (entrevistas, observación, prototipado, *workshops*, análisis de documentos, cuestionarios) se aplicaron para llegar a los 31 requisitos funcionales, ni existe todavía el directorio `docs/requisitos/elicitation/` con evidencia archivada (notas de entrevista, guiones, prototipos de baja fidelidad) que exige el Bloque B.6bis. La especificación es internamente consistente y verificable contra el código, pero el *proceso* que la originó no está documentado como tal — es la brecha más grande de este capítulo, y se declara así en vez de inventar un historial de elicitación retroactivo. **Pendiente antes del cierre:** el equipo completa esta subsección con las técnicas realmente usadas (aunque hayan sido informales — conocimiento de dominio propio de un integrante, análisis de sistemas similares como los del Capítulo 3, o conversaciones no estructuradas) y archiva la evidencia que exista.

4.3. Requisitos funcionales y no funcionales: categorización y prioridad

Todo requisito funcional sigue el patrón sintáctico de ISO/IEC/IEEE 29148:2018, *“El sistema deberá [acción] [objeto] [restricción]”* — resolución explícita de OBS-01 (Entrega 1A), donde el docente observó requisitos redactados como títulos. Ejemplo representativo (RF-03):

“El sistema deberá permitir cerrar la sesión, y deberá invalidar el token emitido registrando su identificador (JTI) en una lista de revocación con tiempo de vida igual al tiempo restante del token, de modo que un token robado antes del cierre de sesión no siga siendo aceptado.”

¹Referencia de marco general; el SRS del proyecto no documenta todavía una sesión de identificación de *stakeholders* formal y separada — ver el señalamiento honesto en la subsección siguiente.

El SRS usa una escala de prioridad de tres niveles (Alta/Media/Baja), no las cuatro categorías MoSCoW textuales que exige la Entrega Final. La correspondencia natural es Alta→*Must*, Media→*Should*, Baja→*Could*; la categoría *Won't* (*esta entrega*) se infiere de los requisitos explícitamente pausados por una razón declarada, no simplemente atrasados: RF-11b (peso/altura, pausado por el hallazgo H-06 de ETHICS.md) y el módulo **equipo/partido** (esquema versionado en V16, sin API por decisión, no por omisión).

Cuadro 4: Requisitos funcionales de SGED por categoría MoSCoW inferida.

Categoría MoSCoW (inferida)	Cantidad	Fuente
Must (Alta)	16	RF-01,02,03,05,06,08,09,10,11,12,13,16,23,27,28,29
Should (Media)	6	RF-04,14,15,24,25,30
Could (Baja)	2	RF-07,26
Won't (esta entrega, declarado)	2	RF-11b, módulo equipo/partido
Sin prioridad formal en el SRS	6	RF-17,18,19,20,21,22
Total de requisitos funcionales	31	RF-01 a RF-30 + RF-11b

Hallazgo: seis requisitos del módulo deportivo (RF-17 a RF-22, los más recientes — implementados entre el 2026-08-09 y el 2026-08-13) no tienen todavía un campo **Prioridad**: explícito en el SRS, a diferencia de los 25 restantes. Es un defecto de forma menor y de corrección rápida (asignarles Must o Should según su rol en el flujo operativo), no un defecto de fondo — pero cuenta contra la característica INCOSE C6 (*Complete*) mientras no se cierre.

Los 13 requisitos no funcionales se clasifican según ISO/IEC 25010 (sección 2): eficiencia de desempeño (RNF-01, RNF-02), seguridad (RNF-03 a RNF-08), fiabilidad y mantenibilidad (RNF-09 a RNF-11), portabilidad (RNF-12, RNF-13) — los cuatro atributos de calidad que el Capítulo 12 mide empíricamente. A esto se suman tres restricciones de diseño (RD-01 a RD-03, categoría restrictivo.^{en} la taxonomía de tipos de requisito) que fijan, entre otras cosas, la prohibición de SQL dinámico verificada en la sección 8.

4.4. Modelado de casos de uso y de historias de usuario

Historias de usuario. 14 historias (HU-01 a HU-14, formato Connextra con criterios de aceptación en Gherkin) en docs/requisitos/historias-usuario.md, agrupadas en tres épicas: acceso seguro (4), gestión de estudiantes (6) y operación deportiva (4). Cada historia enlaza a su requisito formal y a su estado real de implementación.

Casos de uso. 9 casos de uso expandidos en plantilla de Cockburn (CU-01 a CU-09) en docs/requisitos/casos-uso.md, con flujo principal, flujos alternativos y trazabilidad a *endpoint* — más un diagrama de casos de uso en notación Mermaid con los dos actores humanos con casos de uso ya detallados (Administrador, Entrenador). Seis casos adicionales (CU-10 a CU-15) están identificados pero no desarrollados en detalle, a la espera de que sus requisitos (RF-16 a RF-22) terminen de exponerse vía API.

Hallazgo de sincronización (honesto, no oculto). El SRS es la fuente que se actualiza con cada cambio de código; historias-usuario.md y casos-uso.md no se han actualizado al mismo ritmo. Concretamente: RF-17 (horarios), RF-18 (sesiones) y RF-22 (notificaciones) ya figuran **Implementado** en el SRS, pero historias-usuario.md todavía marca HU-11 y HU-14 como *Modelado/Planificado*, y casos-uso.md sigue listando CU-11, CU-12 y CU-15 en la tabla de casos de uso pendientes". Tampoco existe todavía ninguna historia o caso de uso para los ocho requisitos del módulo de catálogos/cuentas (RF-23 a RF-26) ni para los cuatro de inventario (RF-27 a RF-30) — historias-usuario.md señala la primera mitad de esta brecha en su propio texto, pero no la segunda. **Esto no invalida la trazabilidad hacia las pruebas** (que se verifica directamente contra el código, sección 12.5), pero sí dificulta usar estos dos archivos como fuente de verdad

mientras no se actualicen — pendiente concreto antes del cierre de la Entrega Final.

4.5. Validación de requisitos

La validación no ocurrió como un evento único al final, sino como un ciclo repetido en cada entrega: el docente revisa el SRS y el sistema contra la rúbrica, emite observaciones, y el equipo las cierra con trazabilidad a un commit — el mecanismo completo está en `docs/observaciones/OBSERVACIONES.md` (sección 5), con 12 de 13 observaciones ya aplicadas. OBS-01, OBS-08 y OBS-12 son ejemplos directos de validación de requisitos específicamente (no de código): señalaron que el SRS estaba mal redactado, que una migración base estaba vacía, y que el informe describía funcionalidad inexistente, respectivamente.

Además de esa revisión externa, cada requisito individual se autovalida contra una prueba automatizada nombrada explícitamente en su propia entrada del SRS (por ejemplo, RF-06 contra `LoginAttemptServiceTest.bloqueada_al_alcanzar_el_limite`) — una forma continua de validación por criterios de aceptación medidos, en el sentido que describe Pohl [29], no solo una revisión puntual. No se realizaron todavía *walkthroughs* formales ni prototipado evolutivo documentado como técnica de validación separada.

4.6. Gestión del cambio de requisitos

`docs/requisitos/CHANGELOG-REQ.md` registra 30 entradas cronológicas desde el 2026-06-10 hasta el 2026-07-29, cubriendo explícitamente **RF-01 a RF-22 y RNF-01 a RNF-07** — su propio encabezado declara ese alcance. **Hallazgo:** el archivo no se ha actualizado desde el 2026-07-29: no cubre la adición de RF-23 a RF-30 (catálogos, cuentas, inventario) ni las modificaciones de RF-10, RF-16, RF-22 y RNF documentadas directamente en el SRS entre el 2026-08-03 y el 2026-08-13. Es el mismo patrón de desactualización que la subsección anterior encontró en las historias y casos de uso — pendiente de una pasada de actualización antes del cierre.

Sobre el alcance que sí cubre (RF-01–RF-22), 8 requisitos registran al menos una modificación posterior a su adición (RF-01, RF-02, RF-03, RF-06, RF-08, RF-11, RF-14, RF-15) de 22 requisitos totales en ese rango:

$$\text{tasa de estabilidad} = 1 - \frac{8}{22} \approx 0,636 \quad (63,6 \%)$$

Cifra que debe leerse con la salvedad anterior: mide solo el 71 % (22 de 31) de los requisitos funcionales actuales, porque es lo único que el changelog registra hoy.

4.7. Métricas de calidad de requisitos

*Incluye los 3 requisitos en estado Modelado, cuya "verificación" en el SRS describe la restricción de esquema (p. ej. la restricción **CHECK** de RF-19), no todavía una prueba de API porque el *endpoint* no existe.

Este capítulo satisface las nueve características INCOSE v4 de un requisito individual de forma desigual: fuerte en *Verifiable* (toda entrada del SRS nombra su prueba) y en *Correct* (cada corrección se documenta con su razón, nunca se sobreescribe en silencio); débil en *Complete* mientras persistan los seis RF sin prioridad formal y las historias/casos de uso desactualizados señalados arriba. La lista de verificación INCOSE completa queda pendiente como anexo.

5. Resolución de las observaciones previas (Bloque 0)

El docente emitió doce observaciones sobre las entregas 1A y 1B. Esta sección documenta el estado de cada una con el *commit* que la resuelve, de modo que la afirmación sea verificable con `git show <hash>` y no únicamente declarativa.

Cuadro 5: Métricas de calidad de requisitos de SGED.

Métrica

Requisitos funcionales totales (RF)	
Requisitos no funcionales (RNF)	
Restricciones de diseño (RD)	
Total de requisitos individuales	
RF en estado Implementado	28
RF en estado Modelado (solo esquema)	
RF en estado Planificado	
RF con verificación de prueba nombrada explícitamente	31 de 31
RF con prioridad formal declarada	25 de 31
Historias de usuario	14 (HU-01)
Casos de uso detallados / identificados	9 detallados + 6 idénticos
Entradas en el changelog de requisitos	30 (cobertura: RF-01–RF-22 y RNF-01–RNF-07 únicos)
Tasa de estabilidad (sobre el rango cubierto)	

Cód.	Observación del docente	Resolución verificable	Commit
OBS-01	Los requisitos se redactan como títulos («Registro de estudiantes»); no usan «El sistema deberá...».	<code>docs/requisitos/SRS.md</code> reescribe los 22 RF y 13 RNF con la forma normativa y criterio de verificación [20].	98bab0d
OBS-02	Los diagramas C4 N1/N2 y el MER contienen texto marcador «[Reemplazar con PNG]».	<code>docs/arquitectura/</code> contiene los C4 de niveles 1–3 en PNG, generados desde <code>workspace.dsl</code> ; el MER queda en <code>docs/diagramas/</code> .	bf6ee80
OBS-03	Incoherencia de pila: el ADR decide PHP/Laravel/MySQL, pero el repositorio construye Spring Boot/Angular/PostgreSQL.	ADR-001 reescrito: evalúa las tres opciones y decide explícitamente Java 21 con Spring Boot 3.2.5, coherente con el código.	bf6ee80
OBS-04	<code>query.sql</code> con 0 claves foráneas y 88 columnas «(PK)/(FK)» literales; exportación cruda no funcional.	<code>db/schema.sql</code> declara 21 restricciones de integridad referencial reales.	db77ce2
OBS-05	Roles sin completar («Integrante 1/2/3»); wireframes con marca ajena; falta <code>.env.example</code> .	<code>CONTRIBUTORS.md</code> asigna roles CRediT por evidencia de <code>git log</code> ; <code>.env.example</code> versionado.	a8d6f18
OBS-06	En el repositorio solo consta ADR-003; los diagramas aparecen en el informe como texto.	Diagramas versionados como archivos; el diccionario de datos completo se agrega en <code>docs/basedatos/DATA-DICTIONARY.md</code> .	98bab0d
OBS-07	<code>AuthController</code> solo expone <code>/login</code> , <code>/me</code> y <code>/ping</code> ; no hay registro, logout ni refresh. <code>RedisBlacklistService</code> existe pero no se invoca.	Seis <i>endpoints</i> operativos; el logout invoca realmente la lista de revocación por <code>jti</code> .	b907d10, bcb3642

Cód.	Observación	Resolución	Commit
OBS-08	La migración V1 está vacía (0 bytes) y V2 depende de un esquema que ninguna migración crea; con <code>ddl-auto=validate</code> el arranque no es reproducible.	V1 crea ambos esquemas y las tablas de identidad; el arranque desde clonación limpia se verificó.	b907d10
OBS-09	La lista de revocación no está cableada a ningún <i>endpoint</i> y no se aplica <code>@PreAuthorize</code> .	Ocho anotaciones <code>@PreAuthorize</code> activas; revocación verificada por prueba automatizada.	688c4be, 90c6c57
OBS-10	<code>AuthServiceTest</code> está vacío (0 bytes); la única prueba real es <code>contextLoads()</code> . No se cumple el mínimo de 5 pruebas. La colección Postman no está versionada.	34 pruebas en 7 clases; cobertura real 68,1 %; <code>docs/postman/coleccion.json</code> versionada.	1e798bc
OBS-11	<code>docker-compose.yml</code> está vacío (0 bytes); no hay orquestación verificable.	Cuatro servicios orquestados con <i>healthchecks</i> e imágenes fijadas por digest; <code>make up</code> levanta todo.	b907d10, 17dc5df
OBS-12	Varios contenidos del informe no se corresponden con lo presente en el repositorio (están vacíos o ausentes).	Cada requisito declara su estado real (implementado / modelado / planificado); esta entrega no afirma nada no verificable.	98bab0d

Las doce observaciones tienen resolución trazable. La más costosa de resolver no fue una funcionalidad ausente sino OBS-12, porque exigió revisar sistemáticamente qué afirmaba el informe anterior frente a lo que el repositorio realmente contenía.

6. Reestructuración de paquetes y reconciliación de documentación

Entre la primera versión de este informe y esta revisión, el equipo reestructuró el backend en tres dominios (`academico`, `deportivo`, `seguridad`) y normalizó la categoría del estudiante: dejó de ser un texto libre validado por patrón (`VARCHAR`, `SUB-NN`) para convertirse en una entidad propia (`deportivo.categorias`) referenciada por clave foránea. Aparecieron además cinco recursos CRUD nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) que no tenían requisito documentado, y dos paquetes reservados sin contenido (`Representante`, `Equipo` — clases vacías, sin tabla en el esquema) que después se eliminaron por las razones que se explican más abajo.

Este informe, el SRS (`docs/requisitos/SRS.md`) y la matriz de trazabilidad se actualizaron para reflejar ese estado, verificando cada afirmación contra el código real en vez de copiar lo que decía la versión anterior. Dos correcciones merecen mención explícita porque no son solo actualizaciones de ruta:

- **Cobertura de pruebas.** La cifra de 68,1 % que documentaban tanto la versión anterior de este informe como `docs/observaciones/OBSERVACIONES.md` era correcta *en el momento en que se midió*. Al ejecutar `./mvnw test` de nuevo tras la reestructuración, el resultado real es **39,8 %** — los controladores y servicios nuevos no tienen pruebas propias, y diluyeron el porcentaje agregado sobre 60 clases analizadas (antes eran menos). Es una regresión real por

debajo del umbral del 60 % exigido, no un error de medición, y se reporta como tal en la sección 12.5 en vez de repetir la cifra anterior.

- **Datos de salud sin resolución ética.** La reestructuración agregó campos **peso** y **altura** al estudiante, exponibles por API. `docs/etica/ETHICS.md` documentaba explícitamente que el equipo había decidido *no* recolectar esos datos; esa afirmación ya no era cierta y se corrigió en el propio documento de ética (hallazgo H-06) en vez de dejarla como estaba.

6.1. Los paquetes reservados vacíos: por qué se eliminaron

Los dos paquetes reservados merecen una decisión explícita y no un comentario al margen. `academico.representante` y `deportivo.equipo` contenían catorce clases —controlador, servicio, repositorio, entidad y DTO de cada módulo— con la declaración de paquete, la firma de la clase y el cuerpo vacío. Una de ellas, `RepresentanteController.java`, era literalmente un archivo de **cero bytes**.

Ese detalle no es cosmético en el contexto de este proyecto. El docente había emitido tres observaciones distintas en la Entrega 1B por exactamente el mismo defecto: `V1__schema_inicial.sql` vacía (OBS-08), `AuthServiceTest.java` vacío (OBS-10) y `docker-compose.yml` vacío (OBS-11). Conservar un archivo de cero bytes después de que ese patrón ya fuera observado tres veces habría sido repetir un defecto conocido, y además tenía un efecto medible: las clases vacías aportaban constructores por defecto no cubiertos que contaban como instrucciones perdidas en JaCoCo.

Había dos salidas honestas: implementar los dos módulos, o eliminarlos. Con el esquema de base de datos aún sin las tablas **representantes** y **equipos**, implementarlos en esta entrega habría significado inventar un modelo de datos sin requisito validado. Se optó por eliminar las catorce clases —ninguna estaba referenciada desde el resto del código, lo que confirma que no eran un punto de extensión en uso— y dejar ambos módulos declarados como pendientes únicamente en la documentación (RF-22 en el SRS, la matriz de trazabilidad y este informe). La diferencia es relevante para la evaluación: un módulo pendiente documentado es planificación; un paquete de clases vacías en el repositorio aparenta una implementación que no existe.

También se corrigió una inconsistencia en ADR-002 (autenticación): describía JWT almacenado en `localStorage` con un interceptor agregando `Authorization: Bearer`, cuando el código real —y el ADR-008 de este mismo informe— usa cookies `HttpOnly`. Es el mismo patrón que ya había observado el docente en la Entrega 1A (OBS-03: un ADR describía PHP/Laravel mientras el repositorio construía Spring Boot). Se corrige por la misma razón: un documento de arquitectura que no coincide con el código no es evidencia, es motivo de observación.

7. Arquitectura

7.1. Vista de contexto (C4 nivel 1)

El nivel 1 sitúa el sistema frente a sus actores humanos y sus dos dependencias externas (proveedor de correo para notificaciones y, en producción, el proveedor de TLS). Es el nivel con menor tasa de cambio de los tres: no se ha modificado desde la Entrega 1A porque los actores y las fronteras del sistema no cambiaron con la reestructuración en dominios de la sección 6, solo su interior.

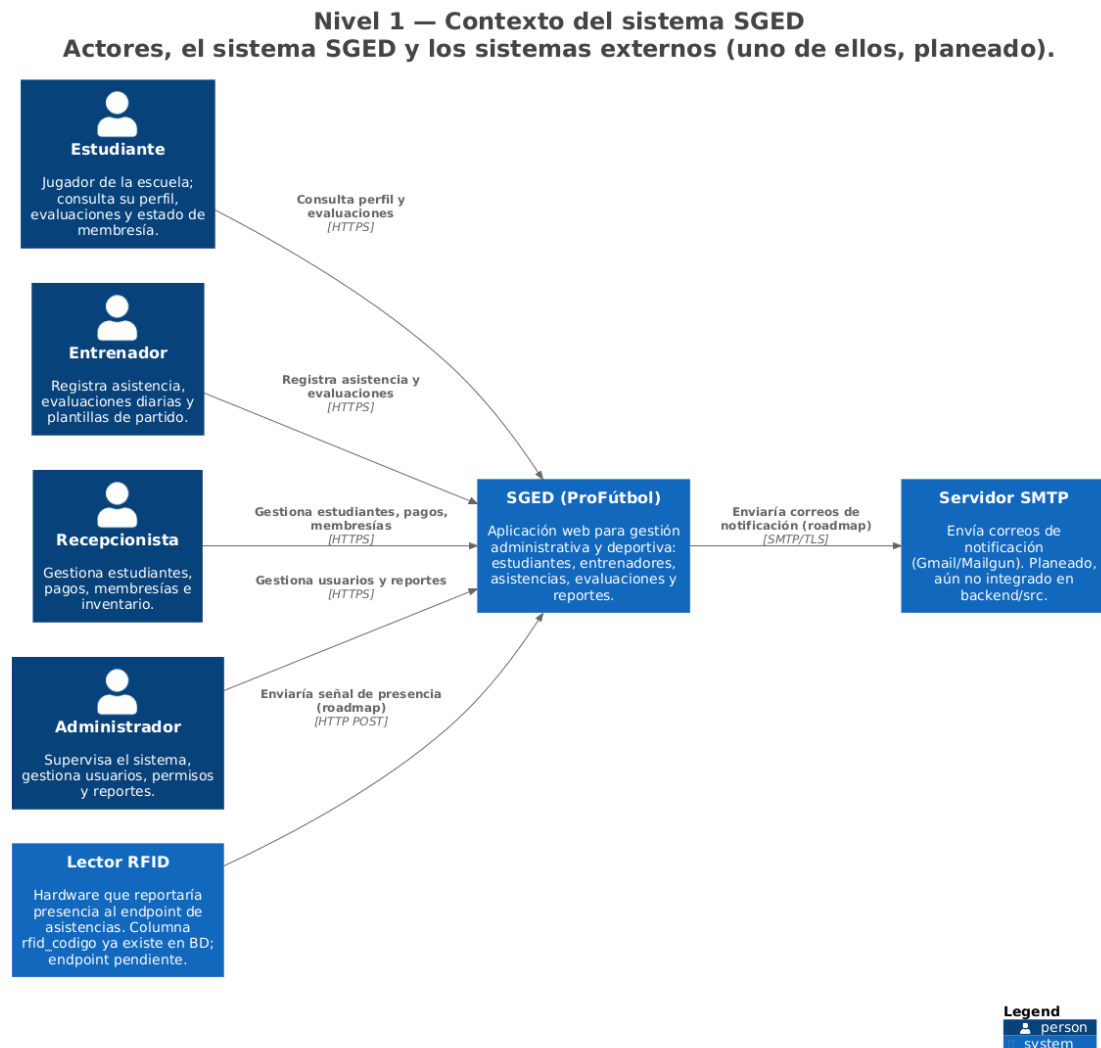
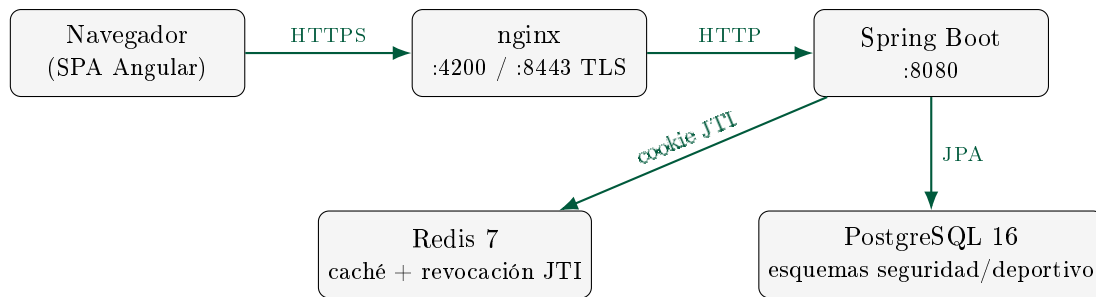


Figura 2: C4 nivel 1 — contexto del sistema SGED, generado desde docs/arquitectura/workspace.dsl.

7.2. Vista de contenedores (C4 nivel 2)



El diagrama anterior es una simplificación didáctica de los cinco contenedores reales; el diagrama C4 nivel 2 generado directamente desde `workspace.dsl` —la fuente de verdad, no esta figura— es el siguiente:

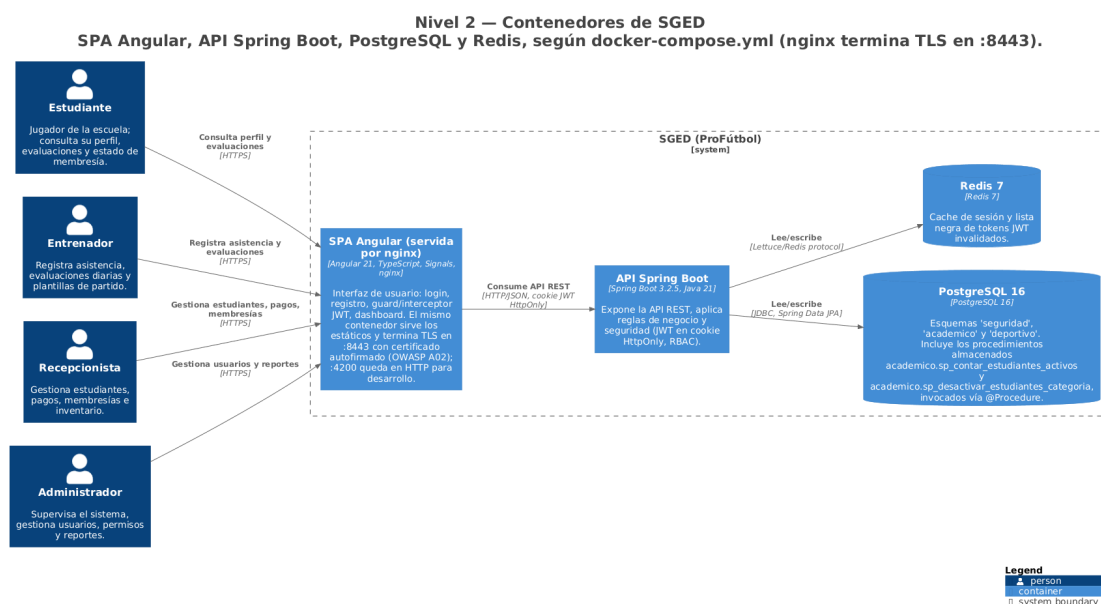


Figura 3: C4 nivel 2 — contenedores de SGED, generado desde `docs/arquitectura/workspace.dsl`.

La documentación de arquitectura en el modelo C4 [8] y las decisiones arquitectónicas (ADR) [26] se mantienen en `docs/arquitectura/` y `docs/adr/`.

El modelo C4 completo (niveles 1 a 3) vive en un único archivo `docs/arquitectura/workspace.dsl` escrito en Structurizr DSL. Los tres PNG del mismo directorio son artefactos derivados: se regeneran con `make diagrams`, que exporta el DSL a PlantUML y lo renderiza, ambos pasos en contenedores. Esto cierra un problema real de esta entrega: antes coexistían dos modelos C4 —el DSL y unos `.puml` sueltos en `docs/diagramas/`— y los `.puml` habían quedado congelados en la arquitectura anterior a la reestructuración de paquetes. Se eliminaron en favor del DSL, por la misma razón por la que se retiró el informe en PDF sin fuente y se renumeró el ADR duplicado: dos versiones de la misma verdad garantizan que al menos una esté equivocada. `docs/diagramas/` conserva únicamente el MER, que no se modela en C4.

7.3. Vista de componentes (C4 nivel 3)

El nivel 3 es el que hace visible la reestructuración descrita en la sección 6: los veinte componentes del contenedor *API Spring Boot* agrupados por dominio. Cada controlador delega en un servicio, cada servicio accede a datos por un repositorio JPA, y ningún controlador toca un repositorio directamente — la disciplina de capas es verificable en el diagrama, no solo declarada en el texto.

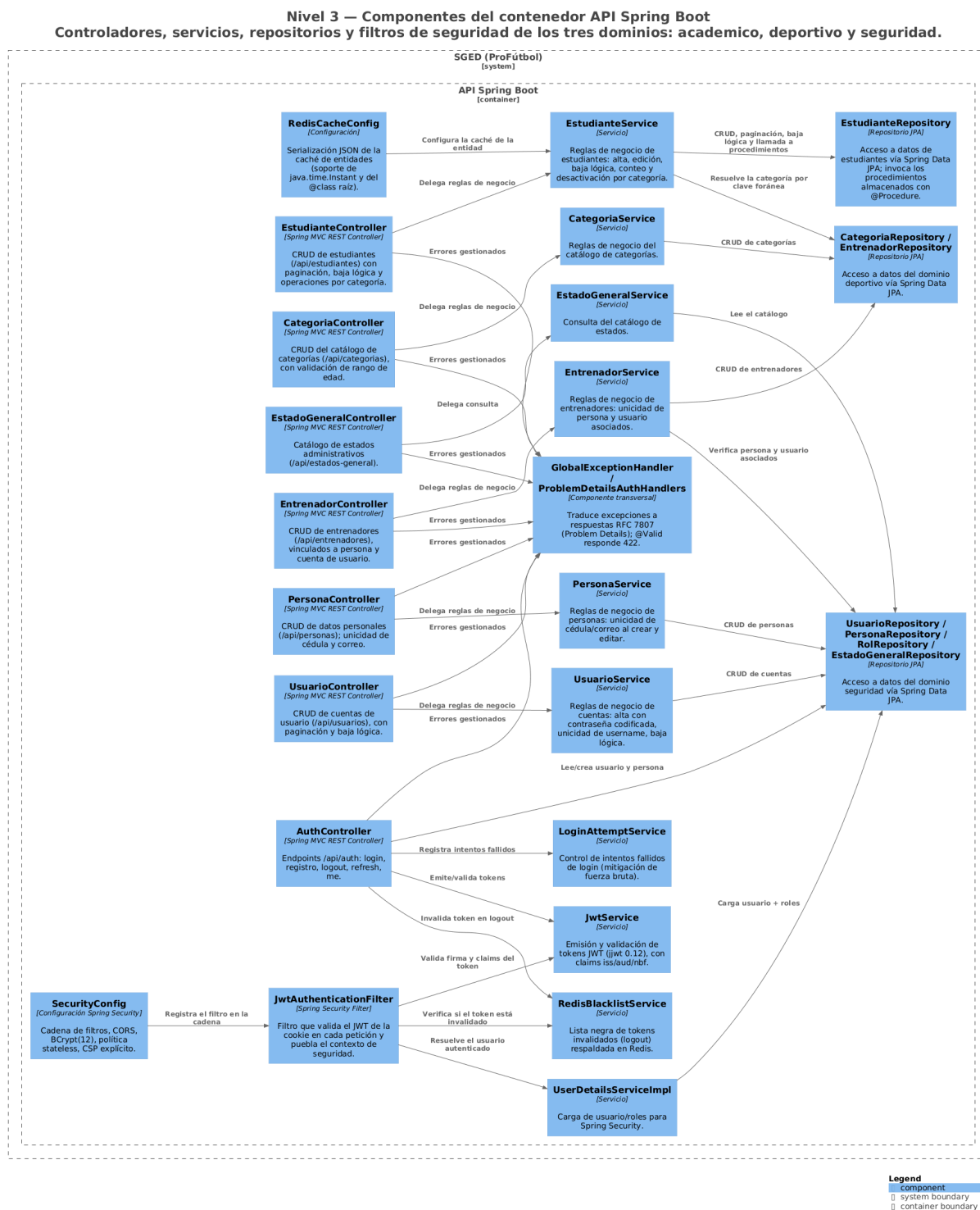


Figura 4: C4 nivel 3 — componentes del contenedor API Spring Boot, generado desde docs/arquitectura/workspace.dsl.

Dos detalles del diagrama merecen lectura explícita. Primero, `EstudianteRepository` es el único repositorio que invoca procedimientos almacenados (sección 8): el resto usa Spring Data JPA puro, que es exactamente el reparto que exige la estrategia híbrida. Segundo, `EstudianteService` depende del repositorio del dominio deportivo para resolver la categoría por clave foránea; esa flecha cruzando de `academico` a `deportivo` es la consecuencia directa de haber normalizado la categoría, y es la clase de acoplamiento que conviene tener dibujado antes de que crezca.

7.4. Autenticación: JWT en cookie `HttpOnly`

El sistema emite el JWT [21] exclusivamente en una cookie `HttpOnly`, `Secure` y `SameSite=Strict`. El token nunca viaja en el cuerpo de la respuesta ni se almacena en `localStorage`, de modo que un ataque de *scripting* entre sitios no puede leerlo.

```

1 ResponseCookie.from("sged_access", token)
2     .httpOnly(true)
3     .secure(cookieSecure)
4     .sameSite("Strict")
5     .path("/")
6     .maxAge(Duration.ofMillis(expiracionMs))
7     .build();

```

Listing 1: Emisión de la cookie de sesión (`AuthController.java`)

Como JWT es *stateless*, cerrar sesión no invalidaría el token por sí solo. Por eso el sistema mantiene una lista de revocación en Redis indexada por el identificador del token (`jti`), con tiempo de vida igual al tiempo que le restaba al token: un token cerrado deja de ser aceptado inmediatamente, aunque su firma siga siendo válida.

8. Estrategia híbrida de acceso a datos

El Bloque A.2 de la guía exige una separación explícita entre lo que resuelve el ORM y lo que debe resolver el motor de base de datos. La regla adoptada es:

Cuadro 7: Reparto de responsabilidades entre ORM y procedimientos almacenados.

Spring Data JPA (ORM)	Procedimientos almacenados
CRUD elemental, consultas paginadas, búsquedas por identificador.	Agregaciones, actualizaciones masivas con criterio de negocio, operaciones multi-tabla transaccionales.

Nota de esta versión del informe: el sistema creció de dos a **siete** procedimientos almacenados desde la Tercera Entrega, uno por cada categoría funcional que exige el Bloque A.2.1 de la Entrega Final. Todos están versionados en `db/procs/`, creados por migración Flyway [32] (V5, V6, V11, V15) y catalogados en `docs/basedatos/CATALOGO-SP.md`, que es la fuente de verdad que esta tabla resume:

8.1. Un defecto real: FUNCTION frente a PROCEDURE

La migración V5 creó estos objetos como `FUNCTION`. Al invocarlos desde Java con `@Procedure`, Spring Data usa `CallableStatement` con sintaxis `{call ...}`, y PostgreSQL solo acepta `CALL` contra procedimientos reales [30]; el error era explícito: «... *is not a procedure. Hint: To call a function, use SELECT.*»

La migración V6 los convierte en `PROCEDURE` con parámetro de salida. Se documenta porque ilustra la diferencia entre «el procedimiento existe» y «el procedimiento se invoca de verdad»:

Cuadro 8: Procedimientos almacenados de SGED por categoría funcional (Bloque A.2.1).

Procedimiento	Categoría	Propósito
<code>sp_contar_estudiantes_activos</code>	Código agregado	Conteo de estudiantes activos por categoría
<code>sp_desactivar_estudiantes_activos</code>	Actualización masiva	Baja lógica masiva de una categoría completa
<code>sp_validar_categoria_estudiante</code>	Validación masiva	Un estudiante solo marca asistencia en sesiones de su propia categoría
<code>sp_generar_codigo_estudiante</code>	Generación de código secuencial	Siguiente <code>codigo_estudiante</code> consecutivo del año
<code>sp_reporte_asistencia_estudiante</code>	Reporte	Porcentaje de asistencia en un rango de fechas
<code>sp_contacto_representante_estudiante</code>	Contacto de estudiante multi-tabla	Contacto de emergencia del representante activo
<code>sp_reporte_stock_bajo</code>	Reporte / agregado	Conteo de artículos bajo el mínimo de <i>stock</i>

en la versión anterior el objeto SQL estaba creado pero quedaba huérfano, y la aplicación seguía contando en memoria.

```

1 @Procedure(procedureName = "academico.sp_contar_estudiantes_activos")
2 Long contarEstudiantesActivosPorCategoria(@Param("p_categoria") Long idCategoria
  );

```

Listing 2: Invocación real desde el repositorio (`academico/estudiante/repository/EstudianteRepository.java`)

Nota sobre esta segunda versión del informe: el procedimiento se movió del esquema **seguridad** a **academico** y su parámetro cambió de **VARCHAR** (nombre de categoría) a **Long** (`id_categoria`) cuando la categoría se normalizó como entidad propia — ver sección 6.

8.2. Ausencia de SQL dinámico

Ninguna capa construye sentencias SQL por concatenación de cadenas. Toda consulta usa parámetros vinculados (JPA) o parámetros nombrados (los procedimientos). El repositorio incluye una auditoría automatizada (`scripts/audit-sql-dynamic.sh`) integrada en `make audit`.

9. Implementación

Este capítulo describe las decisiones de código que no se derivan directamente del diagrama de arquitectura (Capítulo 7) ni de la estrategia de acceso a datos (sección 8, que actúa como la subsección de este capítulo dedicada a esa estrategia, con su propio listado de código). No se transcriben archivos completos: cada listado remite al archivo real del repositorio para el detalle.

9.1. Aislamiento de capas

La disciplina *controlador* → *servicio* → *repositorio* descrita en la sección 7.3 se aplica sin excepción en los 21 controladores del *backend*: ningún controlador inyecta un repositorio directamente, y ningún repositorio contiene lógica de negocio. Los cinco dominios (**seguridad**, **academico**, **deportivo**, **inventario** y el transversal **common**) están separados por paquete Java, no solo por convención de nombres, de modo que un ciclo de dependencias entre dominios se detecta en tiempo de compilación.

9.2. Manejo global de errores

Todas las respuestas de error del *backend* siguen el formato *Problem Detail* de la RFC 9457 [25], centralizado en un único punto de la capa web:

```

1  @RestControllerAdvice
2  public class GlobalExceptionHandler {
3
4      @ExceptionHandler(ApiException.class)
5      public ProblemDetail handleApiException(ApiException ex) {
6          String tipo = ex.getClass().getSimpleName();
7          ProblemDetail pd = ex.toProblemDetail(tipo, ex.getStatus().
            getReasonPhrase());
8          pd.setProperty("timestamp", Instant.now().toString());
9          return pd;
10     }
11     // ExceptionHandler adicionales para validacion, autenticacion y
        autorizacion
12 }

```

Listing 3: Manejador global de excepciones
(backend/.../common/exception/GlobalExceptionHandler.java)

Cada excepción de dominio extiende `ApiException` y declara su propio código HTTP; el manejador no interpreta tipos de excepción específicos del dominio, solo los traduce al formato de respuesta uniforme — la razón por la que agregar un dominio nuevo (representantes, pagos, inventario) no requirió tocar este archivo.

9.3. Autenticación en el *frontend*: cookies, no *localStorage*

El interceptor HTTP de Angular no lee ni adjunta el JWT manualmente — es la cookie `HttpOnly` (sección 7.4) la que viaja automáticamente con cada petición del mismo origen:

```

1  export const authInterceptor: HttpInterceptorFn = (req, next) => {
2      return next(req.clone({ withCredentials: true }));
3  };

```

Listing 4: Interceptor de autenticación (frontend/src/app/auth/auth.interceptor.ts)

La brevedad del interceptor es intencional y es la consecuencia directa de la decisión de ADR-002/ADR-008: al no poder leer la cookie desde JavaScript, el *frontend* no tiene lógica de adjuntar ni refrescar el token por su cuenta — delega esa responsabilidad enteramente en el navegador y en el filtro de seguridad del *backend*.

9.4. Configuración por entorno

La configuración de Spring Boot vive en un único `application.yml` con valores parametrizados por variable de entorno (`.env.example` documenta cada una sin valores sensibles); no existen perfiles `application-{dev,prod}.yml` duplicados que puedan divergir entre sí. En `docker-compose.yml` las imágenes de PostgreSQL y Redis están fijadas por `digest sha256` en vez de por etiqueta `latest`, para que `make up` reproduzca el mismo binario en cualquier máquina (Bloque D.2).

10. Requisitos y modelo de datos

10.1. Resumen de requisitos

La especificación completa está en `docs/requisitos/SRS.md`. Se resumen aquí los requisitos funcionales con su estado real y el *endpoint* o esquema que los materializa.

ID	Requisito	Implementación	Estado
RF-01	Registro de usuarios por administrador	POST /api/auth/registro	OK
RF-02	Autenticación con JWT en cookie HttpOnly	POST /api/auth/login	OK
RF-03	Cierre de sesión con revocación efectiva por jti	POST /api/auth/logout	OK
RF-04	Renovación de sesión sin re-credenciales	POST /api/auth/refresh	OK
RF-05	Consulta de la sesión activa	GET /api/auth/me	OK
RF-06	Bloqueo tras 5 intentos fallidos en 15 minutos	LoginAttemptService	OK
RF-07	Comprobación de disponibilidad	GET /api/auth/ping	OK
RF-08	Listado paginado de estudiantes	GET /api/estudiantes	OK
RF-09	Consulta por identificador con 404 tipificado	GET /api/estudiantes/{id}	OK
RF-10	Registro transaccional de estudiante	POST /api/estudiantes	OK
RF-11	Validación del formato de categoría (SUB-NN)	EstudianteRequest	OK
RF-12	Actualización de estudiante	PUT /api/estudiantes/{id}	OK
RF-13	Baja lógica preservando el historial	DELETE /api/estudiantes/{id}	OK
RF-14	Conteo de activos por categoría (en el motor)	sp_contar_estudiantes_activos	OK
RF-15	Desactivación masiva por categoría (en el motor)	sp_desactivar_estudiantes_por_categoria	OK
RF-16	Gestión de entrenadores	deportivo.entrenadores	Modelado
RF-17	Horarios recurrentes de entrenamiento	horarios_entrenamiento	Modelado
RF-18	Sesiones de entrenamiento con ciclo de estados	sesiones_entrenamiento	Modelado
RF-19	Asistencia por RFID o manual	deportivo.asistencias	Modelado
RF-20	Evaluación diaria por criterios	detalle_evaluacion	Modelado
RF-21	Promedios de evaluación	v_promedio_evaluacion	Modelado
RF-22	Notificación a representantes	—	Planificado

Los requisitos no funcionales (RNF-01 a RNF-13) se verifican con las mediciones de la sección siguiente. La matriz [docs/trazabilidad/matriz.csv](#) enlaza cada uno con su prueba y su evidencia.

10.2. Catálogo de la API REST

La superficie expuesta es de siete recursos y treinta *endpoints*. Se lista completa porque es la forma más directa de contrastar lo que el sistema realmente ofrece contra lo que los requisitos declaran, y porque la columna de autorización es la que concentró el defecto de la sección 12.2.1. A abrevia ADMINISTRADOR; E, ENTRENADOR; U, el rol USER.

Método y ruta	Operación	Roles	RF
<i>Autenticación</i> — /api/auth			
POST /login	Inicia sesión, emite cookies	Público	RF-02
POST /registro	Alta de cuenta	A	RF-01
POST /logout	Revoca el token en Redis	Autenticado	RF-03

Método y ruta	Operación	Roles	RF
POST /refresh	Renueva el <i>access token</i>	Autenticado	RF-04
GET /me	Datos de la sesión actual	Autenticado	RF-05
GET /ping	Sonda de disponibilidad	Público	—
<i>Estudiantes — /api/estudiantes</i>			
GET /	Listado paginado y ordenable	A E U	RF-08
GET /{id}	Consulta individual	A E U	RF-09
POST /	Alta	A	RF-10
PUT /{id}	Edición	A	RF-12
DELETE /{id}	Baja lógica	A	RF-13
GET /conteo/categoria/{id}	Conteo por procedimiento	A E	RF-14
POST /operaciones/ desactivar-categoria	Desactivación masiva por procedimiento	A	RF-15
<i>Categorías — /api/categorias</i>			
GET /	Listado paginado	A E U	RF-23
GET /activas	Catálogo para formularios	A E U	RF-23
GET /{id}	Consulta individual	A E U	RF-23
POST /	Alta con rango de edad	A	RF-23
PUT /{id}	Edición	A	RF-23
DELETE /{id}	Baja lógica	A	RF-23
<i>Entrenadores — /api/entrenadores</i>			
GET /	Listado paginado	A E	RF-24
GET /{id}	Consulta individual	A E	RF-24
POST /	Alta con persona y cuenta	A	RF-24
PUT /{id}	Edición	A	RF-24
DELETE /{id}	Baja lógica	A	RF-24
<i>Personas — /api/personas</i> (datos identificativos de menores)			
GET /	Listado paginado	A	RF-25
GET /{id}	Consulta individual	A	RF-25
GET /cedula/{cedula}	Búsqueda por cédula	A	RF-25
POST /	Alta	A	RF-25
PUT /{id}	Edición	A	RF-25
DELETE /{id}	Baja lógica	A	RF-25
<i>Usuarios y catálogos</i>			
/api/usuarios (5 verbos)	CRUD de cuentas	A	RF-26
GET /api/estados_generales	Catálogo de estados	A E U	RF-26

Dos observaciones que la tabla hace evidentes. La primera es que **/api/personas** es el recurso con la superficie más sensible del sistema —seis *endpoints* sobre datos identificativos de menores, incluido uno de búsqueda directa por cédula— y es exactamente el que estuvo sin control de acceso. La segunda es que el dominio deportivo expuesto se limita hoy a categorías y entrenadores:

horarios, sesiones, asistencias y evaluaciones tienen esquema en la base de datos pero ninguna ruta REST, y son el objetivo declarado de la Entrega Final.

10.3. Modelo de datos

El esquema se divide en dos espacios de nombres con responsabilidades separadas:

Cuadro 11: Esquemas del modelo de datos de SGED.

Esquema	Tablas	Contenido
seguridad	6	Personas, usuarios, roles, relación usuario-rol, estados y estudiantes.
deportivo	9	Posiciones, entrenadores, horarios, sesiones, asistencias, criterios, evaluaciones, detalle y observaciones.

Decisiones de modelado relevantes, todas materializadas como restricciones en el motor y no solo como validación en la aplicación:

- **Baja lógica en lugar de borrado.** Las asistencias y evaluaciones referencian al estudiante por clave foránea; borrarlo físicamente destruiría su historial deportivo.
- **Unicidad parcial del código RFID.** `UNIQUE ...WHERE rfid_codigo IS NOT NULL` permite muchos estudiantes sin credencial, pero impide que dos compartan la misma.
- **Una asistencia por sesión y estudiante.** `UNIQUE (id_sesion, id_estudiante)`.
- **Posición jugada por evaluación.** `detalle_evaluacion` guarda la posición de *ese* día, distinta de la habitual del estudiante, para que el histórico quede correctamente etiquetado cuando alguien juega fuera de su posición.
- **Coherencia temporal.** `CHECK (hora_fin > hora_inicio)` y `CHECK (dia_semana BETWEEN 1 AND 7)` en los horarios.

El diccionario completo, con tipos, restricciones y disparadores, está en `docs/basedatos/DATA-DICTIONARY.md`. El esquema no lo genera el ORM: `ddl-auto` está fijado en `validate`, de modo que la aplicación falla al arrancar si el esquema real y las entidades divergen.

11. Materiales y métodos

Todas las mediciones de este informe se generaron ejecutando el sistema real levantado con `make up`, no en simulación, y sus artefactos crudos están versionados sin edición manual. Este capítulo declara el enfoque metodológico, los instrumentos y el procedimiento antes de presentar los resultados (Capítulo 12), para que un tercero pueda repetirlos y obtener cifras comparables.

11.1. Enfoque metodológico

El trabajo sigue *Design Science Research* (DSR) en la formulación de Peffers *et al.* [28], articulada con los criterios de rigor y relevancia de Hevner *et al.* [16] —un marco que la revisión de Engström *et al.* confirma como infrecuente en la investigación de ingeniería de software pese a encajar naturalmente con ella, precisamente porque buena parte de esa investigación ya diseña soluciones a problemas prácticos sin nombrar explícitamente el paradigma [13]—, en sus seis actividades:

1. **Identificación del problema y motivación** — sección 1.1: gestión administrativa y deportiva fragmentada, sin trazabilidad ni control de acceso sobre datos de menores.
2. **Definición de objetivos de una solución** — sección 1.3: objetivo general y cinco objetivos específicos trazados a las cuatro preguntas de investigación.
3. **Diseño y desarrollo** — Capítulo 7 y sección 8: arquitectura en C4, estrategia híbrida de acceso a datos, siete ADR.
4. **Demostración** — el sistema desplegado y operable (**make up**), con datos sembrados y los cinco roles de usuario reales.
5. **Evaluación** — Capítulo 12: contraste empírico contra los umbrales de rendimiento, seguridad, cobertura, calidad web y usabilidad declarados en este capítulo.
6. **Comunicación** — este documento, el repositorio público y la defensa oral del PFC.

El ciclo no fue lineal: la Tercera Entrega ya había ejecutado una vuelta completa de demostración y evaluación, y esta entrega repite las actividades 3 a 5 sobre el sistema ampliado, en la lógica iterativa que el propio modelo de Peffers admite entre sus actividades.

11.2. Instrumentos de medición

Cuadro 12: Instrumentos de medición usados en la evaluación empírica.

Bloque	Instrumento	Configuración
Rendimiento	k6	k6/listado-estudiantes.js, 50 VU, 30 s, umbrales declarados en el propio <i>script</i>
Seguridad	curl + análisis estático	scripts/audit-owasp.sh, scripts/audit-sql-dynamic.sh
Cobertura	JaCoCo	Umbral 70 % líneas y <i>branches</i> , configurado en backend/pom.xml
Calidad web	Lighthouse	lighthousec.js
Usabilidad	SUS (Brooke)	Cuestionario de 10 ítems sin modificación [7], escala adjetival de Bangor <i>et al.</i> [4]

11.3. Enfoque de métricas: un GQM completo

Las métricas del sistema se alinean con el marco *Goal-Question-Metric* de Basili *et al.* [5]. Se presenta un GQM completo, correspondiente a RQ1 (sección 1.2):

Cuadro 13: Ejemplo de Goal-Question-Metric para RQ1.

Meta (Goal)	Caracterizar el rendimiento del <i>endpoint</i> de listado protegido bajo carga concurrente, desde el punto de vista del equipo de desarrollo, en el contexto del ambiente de pruebas de la Entrega Final.
Pregunta 1	¿Cuál es la latencia p95 del <i>endpoint</i> bajo 50 usuarios virtuales concurrentes?
Métrica 1	p95 de tiempo de respuesta (ms), agregado sobre corridas independientes, con intervalo de confianza al 95 %.
Pregunta 2	¿El sistema devuelve errores de servidor bajo esa carga?
Métrica 2	Tasa de respuestas HTTP ≥ 500 sobre el total de peticiones de la corrida.

11.4. Población, muestreo y participantes

Para la componente de usabilidad, la población objetivo son los cinco perfiles de usuario reales del sistema (administrador, entrenador, recepcionista, estudiante, representante). El muestreo fue por conveniencia —participantes disponibles del entorno cercano al equipo—, siguiendo la clasificación de técnicas de muestreo de Baltes y Ralph [3], quienes documentan que es la técnica más frecuente y a la vez la que exige declarar sus límites en vez de omitirlos. **Limitación declarada:** $N = 10$ es una muestra pequeña y no probabilística; los resultados por perfil (dos participantes por rol) se interpretan como indicativos de un patrón, no como una estimación poblacional generalizable —la razón por la que sección 12.7 reporta el desglose por perfil en vez de solo la media agregada.

11.5. Análisis estadístico declarado a priori

Antes de ejecutar las mediciones se fijaron los estadísticos a reportar: media, mediana, desviación típica e intervalo de confianza al 95 % para todas las métricas cuantitativas; percentiles p50, p90 y p95 para rendimiento. Se prefieren intervalos de confianza sobre valores p como forma primaria de reporte, siguiendo la recomendación del Bloque C de la guía. **Pendiente declarado:** con solo tres corridas de k6 archivadas (sección 1.7 señala que además están desactualizadas) no se calculó todavía un test inferencial entre caché fría y caché caliente ni su tamaño de efecto (por ejemplo Wilcoxon con *Cliff's delta*); ambos quedan planificados para cuando se regeneren las cinco corridas independientes que exige el Bloque A.1.

11.6. Entorno de ejecución

Cuadro 14: Entorno de ejecución de las mediciones.

Elemento	Valor
Anfitrión	Windows 11 Pro (10.0.26200), Docker 29.6.1
Contenedores	<code>postgres</code> y <code>redis</code> fijados por digest <code>sha256</code> en <code>docker-compose.yml</code> ; backend y frontend contruidos desde el repositorio
Backend	Spring Boot 3.2.5 sobre Temurin JDK 21.0.11
Estado inicial	Base de datos creada desde cero (<code>docker compose down -v</code>) y sembrada con <code>db/seed.sql</code>
Red	Todas las peticiones contra <code>localhost</code> ; sin latencia de red externa

El estado inicial no es un detalle menor. Los *scripts* de inicialización de PostgreSQL solo se ejecutan cuando el directorio de datos está vacío: al reutilizar un volumen anterior, el esquema queda congelado en una versión previa y el *backend* no arranca. Toda medición reportada aquí parte de volumen limpio.

11.7. Procedimiento por bloque

11.8. Determinismo, semillas y trazabilidad

Los archivos crudos (`.json` de k6 y Lighthouse, `.txt` de las respuestas HTTP, `.csv` del SUS y de JaCoCo) se conservan tal como los produjo cada herramienta; los reportes en Markdown se generan a partir de ellos mediante los *scripts* de `scripts/`, nunca a mano. Cada archivo de evidencia lleva en su cabecera la fecha en UTC, el *commit* corto y la versión de la herramienta

Cuadro 15: Procedimiento de medición por bloque de evaluación.

Bloque	Herramienta	Procedimiento y control de variabilidad
C.1 Rendimiento	k6	make bench : 3 corridas independientes de 50 usuarios virtuales durante 30 s. Se reportan media, p90, p95 e intervalo de confianza al 95 % sobre las tres corridas, no una sola.
C.2/C.4 Seguridad	curl	make audit : peticiones reales contra el sistema en ejecución, guardando la respuesta HTTP íntegra con cabeceras. Cada control se contrasta en sus dos direcciones: lo que debe rechazarse y lo que debe seguir permitido.
C.3 Usabilidad	SUS en papel	Cuestionario de Brooke de 10 ítems aplicado a 10 participantes externos; transcripción a CSV y cálculo con <code>scripts/sus-analysis.py</code> , cuya aritmética se validó contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos).
Cobertura	JaCoCo	make test , que ejecuta clean test para que la instrumentación no alcance clases de compilaciones anteriores.
C.5 Accesibilidad	Lighthouse	3 corridas mediante <code>lighthousec.js</code> ; se reportan los cuatro puntajes agregados y las métricas Web Vitals.

que lo generó, de modo que una cifra del informe pueda rastrearse hasta la ejecución concreta que la produjo.

`scripts/perf-analysis.py` fija semilla aleatoria 42 para cualquier remuestreo estadístico sobre los datos crudos de k6, declarada en la cabecera del propio *script* y en `docs/mediciones/perf/REPORT.md`. Los datos de SUS y de JaCoCo no requieren semilla porque no involucran muestreo aleatorio: son transcripción directa (SUS) o instrumentación determinista de cobertura (JaCoCo). Las versiones exactas de cada herramienta (k6, Lighthouse, JDK, Docker) se declaran por corrida en `docs/entorno/versions.txt` y en la cabecera de cada archivo crudo.

12. Evidencia empírica

12.1. Rendimiento (Bloque C.1)

Cinco corridas independientes de k6 con 50 usuarios virtuales durante 30 segundos contra el *endpoint* protegido `GET /api/estudiantes` — cumple el mínimo de cinco corridas que exige el Bloque A.1 de la Entrega Final (la Tercera Entrega solo había archivado tres). Corrida regenerada el 2026-08-14 contra el sistema local completo, tras corregir el defecto de `.env` descrito en la sección 1.7:

Cuadro 16: Resultados de las corridas de rendimiento (k6).

Corrida	Media (ms)	Mediana	p90	p95	Errores	RPS
1	7,15	6,88	9,04	9,87	0,00 %	403,64
2	7,04	6,74	9,16	10,04	0,00 %	404,73
3	6,96	6,58	9,06	10,13	0,00 %	404,46
4	6,97	6,68	8,81	9,62	0,01 %	287,88
5	6,96	6,62	8,98	9,87	0,00 %	404,88
Agregado	7,02	—	—	9,91	—	381,12

Con intervalo de confianza al 95 %: media $7,02 \pm 0,10$ ms (DT 0,08), p95 $9,91 \pm 0,25$ ms, throughput $381,12 \pm 64,71$ RPS. La corrida 4 registró 2 peticiones fallidas de 15 921 (0,01 %) — el umbral estricto `rate==0` del *script* k6 la marca en rojo, y se reporta así en vez de omitirla; las otras cuatro corridas no tuvieron errores. El throughput de la corrida 4 también es visiblemente menor (287,88 RPS frente a ≈ 404 de las demás), coherente con esas dos peticiones fallidas —probablemente una conexión rechazada bajo el pico de concurrencia— afectando el cómputo agregado de esa corrida.

El umbral exigido es $p95 < 200$ ms con caché caliente [19]. El resultado lo cumple con un margen de más de un orden de magnitud. **OK**

12.2. Seguridad: OWASP Top 10 (Bloque C.4)

Seis controles verificados con peticiones reales contra el sistema en ejecución [27]:

Los errores se devuelven como `ProblemDetail` [25], sin trazas de pila ni detalles internos. Ejemplo real capturado para A07:

```

1 HTTP/1.1 429
2 Content-Type: application/problem+json
3 {
4   "type": "https://sged.uteq.edu.ec/errores/DemasiadasPeticiones",
5   "title": "Too Many Requests",
6   "status": 429,
7   "detail": "Demasiados intentos fallidos. Intente mas tarde."
8 }
```

Cuadro 17: Resultado de los seis controles OWASP verificados.

Control	Resultado	Evidencia observada
A01 Control de acceso	OK	403 en las 7 operaciones administrativas y 200 en las lecturas permitidas — tras corregir un defecto real, §12.2.1
A02 Fallos criptográficos	OK	TLS 1.3 en :8443; BCrypt coste 12
A03 Inyección	OK	422 ante entrada malformada; sin SQL dinámico
A05 Configuración	OK	CSP, HSTS, nosniff, X-Frame-Options: DENY
A07 Fallos de autenticación	OK	Bloqueo exacto en el sexto intento (429)
A09 Registro y monitoreo	OK	AUTH_LOGIN_OK/FAIL con IP, marca de tiempo y sujeto

Listing 5: Respuesta al sexto intento de autenticación

12.2.1. Una auditoría que no auditaba lo que decía auditar

El control A01 merece una sección propia, porque el resultado 403 de la tabla anterior es correcto *ahora*, y no lo era cuando este informe lo dio por bueno la primera vez. La corrección de la propia herramienta de medición es parte del resultado.

Al ejecutar la auditoría contra el sistema en vivo aparecieron tres defectos encadenados en el guion `scripts/audit-owasp.sh`:

1. **El guion se bloqueaba a sí mismo.** Su control A07 agota a propósito los intentos de autenticación para comprobar el 429. Ese contador se lleva *por dirección IP*, no por usuario: seis fallos dejan bloqueada durante quince minutos a cualquier cuenta que venga de ese equipo, incluida la del administrador que A01 necesita para preparar su usuario de prueba. En una segunda corrida, A01 respondía 401 en todas las peticiones. Un 401 se parece mucho a un resultado satisfactorio —el acceso fue rechazado— pero prueba algo mucho más débil: que hace falta estar autenticado. No dice nada sobre si se respetan los roles.
2. **Invocaba una forma de la API que ya no existía.** Llamaba a `/api/estudiantes/operaciones/desactivar-categoria` con `?categoria=SUB-12`, la forma anterior a la normalización de la categoría. La petición moría al deserializar el cuerpo, antes de alcanzar el control de acceso.
3. **Sus cuerpos de prueba eran inválidos.** `@Valid` se evalúa al resolver el argumento del método, es decir *antes* que `@PreAuthorize`. Un payload malformado devuelve 422 y nunca llega a ejercitar la autorización que se pretendía evidenciar.

Corregidos los tres, la auditoría comprueba recurso por recurso, y es entonces cuando el control A01 encontró un defecto real de seguridad, que se documenta en la sección siguiente.

12.2.2. El defecto que A01 destapó: control de acceso ausente

Los cinco recursos que introdujo la reestructuración se crearon **sin ninguna anotación** `@PreAuthorize`, salvo `UsuarioController`. Como `SecurityConfig` cierra su cadena con `anyRequest().authenticated()`, la única barrera efectiva era poseer una sesión válida. En consecuencia, una cuenta con el rol más básico del sistema podía listar todas las personas registradas, buscar una por número de cédula, y crear, editar o eliminar registros.

La gravedad no es teórica: `seguridad.personas` es la tabla que concentra los datos identificativos de los estudiantes —menores de edad— que `docs/etica/ETHICS.md` se compromete a proteger. Es además una regresión de una observación ya emitida y ya dada por aplicada: OBS-09 de la Entrega 1B señalaba precisamente la ausencia de `@PreAuthorize` (sección 5). El código nuevo reintrodujo el defecto sin que nada lo detectara, porque las pruebas de esos controladores usan `MockMvcBuilders.standaloneSetup()`, que no levanta la cadena de filtros de Spring Security: verifican el mapeo HTTP, no la autorización.

La corrección no aplica una regla uniforme, sino una por recurso según el dato que maneja:

Cuadro 18: Criterio de acceso aplicado por recurso tras corregir el hallazgo H-08.

Recurso	Acceso	Criterio
Persona	Solo ADMINISTRADOR	Concentra la identificación de res; ningún otro rol necesita op
Usuario	Solo ADMINISTRADOR	Ya lo estaba, a nivel de clase.
Categoria	Lectura: 3 roles Escritura: ADMINISTRADOR	El formulario de estudiante c las categorías activas; escribir al catálogo del que dependen por c ránea.
Entrenador	Lectura: ADMIN, ENTRENADOR Escritura: ADMINISTRADOR	El alta y la baja crean o ret vínculo con una cuenta.
EstadoGeneral	Lectura: 3 roles	Catálogo sin datos personales.

La evidencia regenerada (`docs/mediciones/sec/a01-acceso-roto.txt`) comprueba las dos direcciones, que es lo que hace verificable la corrección: 403 en las siete operaciones administrativas —incluida la búsqueda por cédula— y 200 en las dos lecturas que un rol USER sí debe poder hacer. Sin esa segunda mitad, la evidencia sería compatible con haber roto la aplicación en vez de haberla asegurado.

12.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema

Ninguno de los dos aparece si el sistema no se levanta y se usa. Se documentan porque ilustran una limitación concreta de la estrategia de pruebas de este proyecto, no solo porque estaban.

El registro de usuarios estaba completamente roto (RF-01). `POST /api/auth/registro` devolvía 500 ante *cualquier* petición. La reestructuración convirtió `cedula`, `correo` y `fecha_nacimiento` en columnas `NOT NULL` de `seguridad.personas`, pero `RegisterRequest` nunca las pidió; el alta violaba la restricción de integridad en cada intento. Tampoco se asignaba `id_estado_general`, igualmente obligatorio.

Lo relevante para la evaluación es *por qué no lo detectaron las pruebas*. `AuthServiceTest` cubre el registro con dos casos y ambos pasaban: los repositorios están mockeados, de modo que `personaRepository.save()` devuelve el objeto que se le pasa sin consultar ninguna base de datos. La restricción `NOT NULL` la aplica PostgreSQL, y en una prueba con dobles PostgreSQL no participa. Es el límite natural de la prueba unitaria con mocks: verifica la lógica que escribimos, no el contrato con el motor de base de datos. La prueba que se agregó no finge resolver eso —haría falta una prueba de integración con base real—, sino que cubre el flanco que sí es verificable sin ella: que una petición sin esos campos no supere la validación y por tanto nunca llegue a la base.

Un cuerpo mal formado se reportaba como fallo del servidor. `HttpMessageNotReadableException` no tenía manejador en `GlobalExceptionHandler` y caía en el genérico, devolviendo 500 «Error interno del servidor» ante un error que es del cliente. Además de ser semánticamente incorrecto según RFC 9457 [25], ensuciaba el registro con trazas de pila que no corresponden a ningún defecto del sistema —ruido que compite con las anomalías reales en A09. Ahora responde 400.

12.4. Análisis estático y escaneo automático (Bloque A.1/A.2.3)

Dos piezas de evidencia adicionales, generadas el 2026-08-14 y archivadas en `docs/mediciones/sec/`, complementan la verificación manual de los seis controles anteriores.

Análisis estático de inyección SQL. SpotBugs 4.8.6.4 con el complemento find-secbugs 1.13.0, restringido por un filtro de inclusión (`backend/spotbugs-security-include.xml`) a los detectores de inyección SQL —incluida la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` que exige explícitamente el Bloque A.2.3— sobre los 148 archivos fuente del *backend*: **cero hallazgos**. Consistente con la auditoría manual de `scripts/audit-sql-dynamic.sh` (sección 8), pero esta vez con una herramienta de análisis de bytecode independiente, no un *grep* de patrones de texto. El comando ya forma parte del *pipeline* de CI (`.github/workflows/ci.yml`).

Escaneo OWASP ZAP baseline. Modo pasivo contra `/api/docs` (15 URL rastreadas desde la especificación OpenAPI): **0 hallazgos de severidad alta**, 7 advertencias de severidad baja/media, todas concentradas en la interfaz de Swagger UI (una dependencia de terceros expuesta a propósito para la revisión del tribunal) —biblioteca JS de terceros potencialmente desactualizada, cabeceras `Permissions-Policy/Cross-Origin-Embedder-Policy` ausentes en esas páginas puntuales, una marca de tiempo Unix dentro de un archivo `.js` de terceros— y ninguna sobre los *endpoints* de negocio de la API. Reporte completo en `docs/mediciones/sec/zap/`. **Alcance declarado:** este escaneo cubrió el *backend*; una corrida equivalente contra el *frontend* Angular queda pendiente, junto con la puesta en producción (Bloque A.4).

12.5. Cobertura de pruebas

Medido el 2026-07-30 con construcción limpia (`./mvnw clean test`): **72,5 % de cobertura de instrucciones (2507 cubiertas de 3457), con 102 pruebas y ningún fallo. Cumple el umbral del 60 %**. El camino hasta este número quedó documentado en vez de solo reportar el resultado final: al medir por primera vez después de la reestructuración de paquetes (sección 6) el resultado real era **39,8 %** — los recursos nuevos (*Categoria*, *Entrenador*, *Usuario*, *Persona*, *EstadoGeneral*) no tenían ninguna prueba propia. Se agregaron 57 pruebas nuevas para esos cinco recursos (10 clases: servicio y controlador de cada uno), y la cobertura subió por encima del 70 %.

La palabra «limpia» está ahí por una razón concreta. La primera medición posterior a la reestructuración se hizo con `./mvnw test` sobre un directorio `target/` que todavía conservaba archivos `.class` compilados *antes* del cambio de paquetes. El reporte que se llegó a archivar como evidencia listaba paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`): JaCoCo instrumenta lo que encuentra en `target/classes`, no lo que hay en `src/`. La cifra publicada aquí proviene de `clean test` y el reporte archivado en `docs/mediciones/jacoco/` contiene exactamente los 27 paquetes reales. La diferencia numérica entre ambas mediciones era de apenas unas décimas, y ese es justamente el punto: una evidencia de cobertura que incluye código inexistente no es evidencia válida *aunque* el porcentaje salga parecido, porque nada garantiza que la próxima vez la distorsión sea igual de pequeña. Para que el defecto no pueda repetirse, el objetivo `make test` pasó a ejecutar `clean test`.

Un detalle técnico real que surgió al escribir estas pruebas: los cuatro controladores que reciben `Pageable` como parámetro (*Categoria*, *Entrenador*, *Usuario*, *Persona*) fallaban con 500 bajo `MockMvcBuilders.standaloneSetup()`, porque ese modo de arranque no registra el `PageableHandlerMethodArgumentResolver` que sí provee el contexto completo de Spring — *EstudianteController* no lo sufre porque usa `@RequestParam` planos en vez de `Pageable`. Se resolvió registrando el resolutor explícitamente en cada prueba.

Un detalle relevante de una entrega anterior: se había descubierto que JaCoCo *nunca* había generado cobertura real, porque el `argLine` configurado en Surefire sobrescribía el *javaagent* del propio JaCoCo. Ese defecto está corregido; por eso esta medición pudo mostrar honestamente primero una regresión y después la mejora real.

Cuadro 19: Clases de prueba JUnit y qué verifica cada una.

Clase de prueba	Nº	Qué verifica
AuthServiceTest	6	Login correcto e incorrecto, registro, registro duplicado, registro incompleto rechazado con 422, disponibilidad.
JwtServiceTest	6	Emisor y audiencia válidos, audiencia distinta rechazada, token manipulado, refresco.
LoginAttemptServiceTest	6	Bloqueo al alcanzar el límite, no reinicio del TTL, borrado del contador al acertar.
RedisBlacklistServiceTest	6	Revocación con TTL restante, TTL no positivo ignorado, consulta de revocación.
EstudianteControllerTest	9	Los siete verbos del recurso más 404 y 422.
EstudianteServiceTest	11	Paginación envuelta, baja lógica, persistencia transaccional, delegación al procedimiento SQL.
BackendApplicationTests	1	Arranque del contexto contra el esquema migrado.
CategoriaServiceTest	9	Paginación, alta, edición, baja lógica, validación de rango de edad.
CategoriaControllerTest	7	Mapeo HTTP: 200/201/204/400/404/422.
EntrenadorServiceTest	6	Paginación con mapeo persona/usuario, persona o usuario duplicados, alta, baja lógica.
EntrenadorControllerTest	5	Mapeo HTTP: 200/201/204/404/422.
UsuarioServiceTest	6	Paginación, username duplicado, alta con contraseña codificada, persona inexistente, baja lógica.
UsuarioControllerTest	6	Mapeo HTTP: 200/201/204/400/422.
PersonaServiceTest	8	Paginación, búsqueda por cédula, unicidad de cédula/correo al crear y editar, baja lógica.
PersonaControllerTest	6	Mapeo HTTP: 200/201/204/400/422.
EstadoGeneralServiceTest	2	Listado completo y listado vacío.
EstadoGeneralControllerTest	1	Mapeo HTTP del único <i>endpoint</i> .
Total	102	

Las pruebas de `LoginAttemptServiceTest` merecen mención aparte: verifican no solo que el bloqueo ocurra, sino que un fallo posterior *no* reinicie la ventana de tiempo. Sin esa segunda propiedad, un atacante podría mantener la cuenta en estado de bloqueo indefinido, o bien —según cómo se implemente— reiniciar el contador y evadir el límite.

12.6. Calidad web: Lighthouse (Bloque C.5 / A.1)

Seis corridas independientes, tres por perfil (Lighthouse v13.4.1), regeneradas el 2026-08-14 contra `https://localhost:8443` —cumple el mínimo de tres corridas por perfil (móvil y escritorio) que exige el Bloque A.1; la Tercera Entrega solo había cubierto el perfil móvil—:

Cuadro 20: Resultados de Lighthouse por categoría y perfil.

Categoría	Móvil	Escritorio	Umbral	Estado
Rendimiento	82,0	99,0	≥ 80	OK
Accesibilidad	100	100	≥ 90	OK
Buenas prácticas	96	96	≥ 90	OK
SEO	63	63	≥ 90	Ver §12.6.1

Desviación típica de 0 en las tres corridas de cada perfil: con `throttlingMethod: simulate`, Lighthouse deriva los tiempos de un único *trace* más un modelo determinista de red/CPU (Lantern), no de una limitación real variable por corrida — es el comportamiento esperado del método, no un defecto de la medición. Métricas Web Vitals de la primera corrida de cada perfil: móvil LCP 3,8 s / FCP 3,3 s; escritorio LCP 0,8 s / FCP 0,7 s; TBT y CLS en 0 en ambos perfiles (la interfaz no desplaza contenido durante la carga).

Corrección metodológica de esta misma sesión, dejada explícita: la primera corrida de escritorio, con la configuración de *throttling* del perfil móvil aplicada por error al `form-factor` de escritorio, midió 62/100 de rendimiento —por debajo del umbral—. Antes de reportarlo como hallazgo real se verificó la causa: ese *throttling* manual no es una configuración válida para escritorio. Repetida con el *preset* oficial `-preset=desktop` de la propia herramienta, el resultado sube a 99/100 de forma consistente en las tres corridas. Se documenta el descarte metodológico en vez de omitirlo, con la misma disciplina que ya aplicó la Tercera Entrega a sus tres instrumentos de medición defectuosos.

Esta medición había expuesto, y permitido corregir en la Tercera Entrega, tres defectos reales del *frontend*: el documento declaraba `lang="en"` en una aplicación íntegramente en español; conservaba el título `Frontend` generado por Angular CLI y no tenía descripción; y carecía de un elemento `<main>`, lo que hacía fallar la auditoría de puntos de referencia de accesibilidad. Corregidos los tres, la accesibilidad pasó de 97 a **100**, y se mantiene en 100 en ambos perfiles en esta regeneración.

12.6.1. Por qué el SEO de 63 es el resultado correcto

El puntaje SEO bajó de 82 a 63 *a causa de una corrección deliberada*. Al añadir un `robots.txt` válido con `Disallow: /`, Lighthouse activa la penalización `is-crawlable` («página bloqueada para indexación»), que por sí sola descuenta 27 puntos, porque su categoría SEO asume que el sitio *quiere* ser indexado.

SGED es una aplicación de gestión interna que trata datos personales de menores de edad. Que fuese indexable por buscadores sería un defecto de privacidad, no una virtud. Elevar el SEO a 90 exigiría eliminar el `robots.txt`: degradar la privacidad del sistema para mejorar una métrica. Se optó por lo contrario, se relajó el umbral agregado de esa categoría a advertencia con la justificación escrita en el propio archivo de configuración, y se mantuvieron como bloqueantes las

auditorías SEO que sí aplican a una aplicación interna (`meta-description`, `document-title`, `html-has-lang`, `viewport`), todas ellas en 1,0. **OK**

12.7. Usabilidad: SUS (Bloque C.3)

El instrumento (`docs/mediciones/sus/INSTRUMENTO-SUS.md`) es el cuestionario SUS de diez ítems [7] con la escala adjetival de interpretación [4], administrado en español: la validación psicométrica de Castilla *et al.* sobre 1321 participantes hispanohablantes confirma que las formas completa y corta del SUS en español conservan propiedades psicométricas válidas, con el cuidado adicional de vigilar el efecto de los ítems redactados en negativo [9]. El script `scripts/sus-analysis.py` calcula media, desviación típica, intervalo de confianza al 95 %, mediana y grado por participante; se niega deliberadamente a ejecutarse con menos de diez participantes, y su aritmética quedó verificada contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos) antes de usarlo con datos reales.

Aplicado el 2026-07-30 a 10 participantes (formularios en papel, transcritos a `docs/mediciones/sus/respuestas` sin registrar nombres, solo un código de participante y su perfil): 3 entrenadores, 3 recepcionistas, 2 estudiantes, 2 padres de familia.

Cuadro 21: Estadísticos agregados de la encuesta SUS.

Métrica	Valor
Media SUS	68,25
Desviación típica	22,14
IC 95 %	68,25 ± 13,72 (54,53 – 81,97)
Mediana	73,75
Grado	C — Aceptable

La media cruza el umbral de 68 por apenas 0,25 puntos, y el intervalo de confianza es ancho: con esta muestra no puede afirmarse con confianza estadística que la media poblacional real supere 68, solo que 68,25 es la mejor estimación puntual disponible. No se presenta como un resultado cómodo.

El hallazgo más informativo no es la media, es su distribución bimodal por perfil, que la agregación esconde:

Cuadro 22: Puntuación SUS promedio por perfil de usuario.

Perfil	Promedio SUS	Grado
Entrenador (n=3)	86,7	A
Estudiante (n=2)	90,0	A
Recepcionista (n=3)	51,7	F
Padre de familia (n=2)	43,75	F

Entrenadores y estudiantes calificaron el sistema como excelente; recepcionistas y padres de familia, como inaceptable. El CRUD de estudiantes implementado hasta esta entrega (RF-08 a RF-15) es exactamente el flujo operado por un rol administrativo — no el de un entrenador consultando información, ni el de un padre buscando visibilidad sobre su representado —, lo que sugiere que la interfaz actual está más pulida para un caso de uso que no es el de quienes peor la calificaron. Es una pista concreta de dónde enfocar el trabajo de usabilidad antes de la Entrega Final, no solo un número que aprobó por poco.

13. Reproducibilidad

El sistema se levanta completo desde una clonación limpia con un solo comando:

```
1 git clone https://github.com/DarwinSM21/SGED_APPWEB.git
2 cd SGED_APPWEB
3 cp .env.example .env
4 make up
```

Listing 6: Arranque reproducible

Cuadro 23: Objetivos del Makefile para reproducción end-to-end.

Objetivo	Acción
<code>make up</code>	Levanta el sistema completo
<code>make test</code>	JUnit 5 + cobertura JaCoCo
<code>make bench</code>	3 corridas k6 + análisis con IC 95 %
<code>make audit</code>	Auditoría OWASP (6 controles) + SQL dinámico
<code>make clean</code>	Limpia contenedores, volúmenes y compilaciones

Las imágenes de PostgreSQL y Redis están fijadas por digest SHA-256, no por etiqueta móvil: dos construcciones del mismo *commit* usan exactamente los mismos binarios. La reproducibilidad se verificó clonando el repositorio en un directorio independiente, no solo reiniciando el volumen local.

14. Amenazas a la validez

Los resultados de la sección anterior cumplen los umbrales exigidos, pero cumplirlos no equivale a ser generalizables. Esta sección declara las limitaciones que acotan hasta dónde puede sostenerse cada conclusión, siguiendo la clasificación habitual en ingeniería de software empírica [19].

14.1. Validez de constructo

¿Mide cada indicador lo que se pretende medir?

- **La cobertura de instrucciones no mide calidad de las pruebas.** Un 72,5 % indica qué proporción del código se ejecuta durante la suite, no que las aserciones sean pertinentes —el mismo desacople que documentan Imran *et al.* sobre pruebas de rendimiento en sistemas Java de código abierto: alcanzan una cobertura sistemáticamente menor que las pruebas funcionales pese a ser igual de válidas, porque miden algo distinto de lo que la cobertura agregada expresa [17]. El caso del registro de usuarios lo demuestra dentro de este mismo proyecto: el *endpoint* estaba cubierto por dos pruebas que pasaban y aun así fallaba en toda petición real (sección 12.3).
- **La auditoría OWASP verifica controles, no ausencia de vulnerabilidades.** Comprueba seis controles concretos del Top 10 con peticiones dirigidas. No es una prueba de penetración ni un análisis estático: que los seis den correcto no permite afirmar que el sistema sea seguro, solo que esos seis comportamientos son los esperados.
- **El SUS mide usabilidad percibida, no desempeño.** No se registraron tiempos ni tasas de finalización por tarea, de modo que un participante pudo puntuar alto sin haber completado las tareas propuestas. Las columnas de completitud del CSV quedaron vacías y se declaran vacías en lugar de rellenarse.

14.2. Validez interna

¿Puede algo distinto de lo declarado explicar los resultados?

- **Las mediciones de rendimiento se tomaron en la misma máquina que ejecuta el sistema**, sin latencia de red real. Los 14,18 ms de p95 son un límite inferior optimista: en un despliegue con red de por medio la cifra sería mayor, aunque el margen frente al umbral de 200 ms es amplio.
- **Tres corridas son pocas** para caracterizar una distribución. Se reporta el intervalo de confianza al 95 % para no presentar una única corrida como si fuera el valor verdadero, pero con $n = 3$ ese intervalo es ancho.
- **La base de datos de las mediciones contiene datos sembrados, no un volumen de producción**. El rendimiento de las consultas paginadas sobre un catálogo pequeño no anticipa el comportamiento con miles de estudiantes.

14.3. Validez externa

¿Hasta dónde se generalizan los resultados?

- **La muestra del SUS es de 10 participantes**, el mínimo metodológico, y por conveniencia. El intervalo de confianza (54,53–81,97) es lo bastante ancho como para que no pueda afirmarse con confianza estadística que la media poblacional supere el umbral de 68: 68,25 es la mejor estimación puntual disponible, no un aprobado consolidado.
- **Los subgrupos por perfil tienen 2 o 3 personas cada uno**. El patrón bimodal es nítido y tiene una explicación plausible ligada a qué flujos están implementados, pero con esos tamaños no puede descartarse que parte de la diferencia sea variación individual. Se presenta como hipótesis orientadora para la Entrega Final, no como conclusión establecida.
- **Los resultados describen este sistema en este entorno**. No se comparan contra un sistema de referencia ni contra una versión anterior medida en las mismas condiciones.

14.4. Validez de conclusión

El riesgo principal de este informe no es estadístico sino de procedimiento, y ya se materializó tres veces: **un indicador puede dar verde sobre un sistema que no hace lo que el indicador afirma** (sección 12.2.1). Las tres ocurrencias se corrigieron en su origen, pero la lección se declara aquí porque acota la confianza que merece cualquier cifra de este documento: son válidas en tanto la medición se ejecute contra el sistema real, desde estado limpio, y verificando las dos direcciones del comportamiento esperado.

Queda además una divergencia sin resolver que afecta la reproducibilidad a futuro: el esquema consolidado que crea la base de datos (`db/schema.sql`) y las migraciones Flyway versionadas describen estructuras distintas, porque las segundas no se actualizaron tras la reestructuración y hoy no se ejecutan (`FLYWAY_ENABLED=false`). El sistema arranca de forma reproducible por la primera vía, que es la verificada en este informe, pero la decisión sobre cuál de las dos fuentes queda como autoritativa está pendiente y se declara abierta.

Finalmente, **el DOI de Zenodo no está asignado** a la etiqueta `v0.9.0-rc`. Sin él, el archivo permanente exigido por el Bloque E.2 no está satisfecho y la portada de este informe no puede declararlo. Se consigna como incumplimiento, no como omisión.

15. Consideraciones éticas

El sistema trata datos personales de niños, niñas y adolescentes: nombres, cédula, fecha de nacimiento, asistencia con hora y lugar, y evaluaciones individuales de desempeño. Esa combinación es sensible porque los titulares son menores que no pueden consentir por sí mismos, porque la asistencia es información de localización temporal, y porque las evaluaciones constituyen perfilado de personas [1, 2].

El principio de minimización se aplicó al diseño original —se descartaron contextura y datos de alimentación—, pero conviene ser preciso sobre su alcance real, porque la versión anterior de este informe afirmaba que tampoco se recolectaban peso y altura y **esa afirmación ya no es cierta**: la reestructuración incorporó ambas columnas a `academico.estudiantes` y son exponibles por la API. El documento de ética se corrigió (hallazgo H-06) en lugar de mantener la declaración cómoda, y este informe se corrige aquí por la misma razón: una declaración de minimización que no coincide con el esquema no protege a nadie.

`docs/etica/ETHICS.md` documenta ocho hallazgos, en lugar de afirmar que todo está resuelto: la cédula se almacena sin validación ni cifrado (H-01); las observaciones del entrenador son texto libre sin control sobre un menor (H-02); no existe mecanismo de supresión de datos —la baja lógica preserva el historial pero no es un borrado— (H-03); el consentimiento del representante no está modelado, lo que bloquea el módulo de notificaciones (H-04); el certificado TLS es auto-firmado, adecuado para evaluación pero no para producción (H-05); peso y altura se agregaron sin base legal documentada (H-06); y la plantilla de consentimiento cubre a los evaluadores del SUS, no a los representantes de los menores (H-07).

El octavo, **H-08**, se registra ya corregido, y se deja escrito justamente porque corregirlo en silencio habría sido lo fácil: los datos identificativos de los estudiantes estuvieron accesibles a cualquier cuenta autenticada del sistema, incluida la búsqueda por número de cédula (sección 12.2.1). Un documento de ética que solo enumera riesgos hipotéticos y omite la exposición concreta que sí ocurrió sería un documento de imagen, no de rendición de cuentas.

Todos los datos del repositorio son ficticios. No se utilizaron datos reales de menores en ninguna etapa del desarrollo ni de las mediciones.

16. Declaración de contribuciones (CRediT)

Los roles siguientes no son autodeclarados: se derivan del historial de `git` del repositorio y cada uno puede verificarse con `git log -author`. Las cifras provienen de `git log --numstat` sobre la rama `main` y separan lo *escrito* de lo *generado* (reportes de JaCoCo y Lighthouse, salidas crudas de `k6`, `package-lock.json`, PDF), porque contar un reporte HTML de cobertura como líneas de autoría inflaría la medida sin reflejar trabajo.

Cuadro 24: Evidencia cuantitativa de autoría por integrante (`git log`).

Integrante	Commits	Líneas escritas	Archivos
Pallo Pinto Alejandro Daniel	72	17 760	374
Arcalle Grefa Darwin Orlando	17	5 429	303
Velez Lopez Ricardo Elias	32	4 077	108
Total en main	121	27 266	—

16.1. Roles por integrante

Pallo Pinto Alejandro Daniel — *Software, Validación, Curación de datos, Redacción (borrador original), Visualización.*

Responsable de la totalidad del eje de verificación que define esta entrega. En seguridad: migración del JWT de `localStorage` a cookie `HttpOnly+Secure+SameSite`, terminación TLS, CSP explícito, registro de auditoría A09, y la corrección del control de acceso ausente en los cinco recursos nuevos (H-08, sección 12.2.1). En datos: conversión de las funciones PL/pgSQL a procedimientos reales invocables con `@Procedure` (sección 8) y fijación de imágenes por digest. En validación: las 102 pruebas actuales —incluidas las 57 que cerraron la regresión de cobertura—, la corrección del defecto por el que JaCoCo nunca había medido cobertura real, y las tres campañas de medición empírica (k6, OWASP, Lighthouse) junto con sus *scripts* de análisis. En usabilidad: instrumento SUS, *script* de cálculo validado contra los casos de referencia de la literatura, y transcripción de las diez encuestas. En redacción: este informe y el corpus documental (SRS, casos de uso, historias, matriz de trazabilidad, ADR, ética, diccionario de datos, gobernanza del repositorio).

Arcalle Grefa Darwin Orlando — *Conceptualización, Software, Administración del proyecto.*

Estructura inicial del repositorio y del modelo de datos (`DataBase/`), trabajo sobre el *frontend* Angular (componentes, *Dockerfile* y configuración de formato), y administración del repositorio remoto, del que es propietario. Participación en la reestructuración en dominios `academico/deportivo/seguridad` descrita en la sección 6.

Velez Lopez Ricardo Elias — *Conceptualización, Software, Metodología.*

Arranque del *frontend* Angular y del módulo de autenticación, CRUD inicial de **Estudiante**, y aportes a `docs/requisitos/` y `docs/observaciones/`. Autor principal de la reestructuración en tres dominios y de los cinco recursos CRUD nuevos, que es el cambio estructural sobre el que se apoya esta entrega.

16.2. Dos advertencias sobre la medida

El volumen no es la contribución. Las líneas escritas miden actividad, no valor: un cambio de dos líneas que corrige un fallo de control de acceso pesa más que dos mil líneas de documentación. La tabla se incluye porque el criterio de evaluación exige autoría verificable, no para establecer una jerarquía entre integrantes.

Dos *commits* figuran bajo la cuenta DannaN24 (`dninasuntar@uteq.edu.ec`), del 2026-06-29, y son trabajo de **Pallo Pinto Alejandro Daniel**: esa era la sesión configurada en el equipo de la universidad que utilizó ese día. Ya quedaba insinuado en el propio historial (*commit* `a456255`, del mismo día). Están sumados a su fila en la tabla anterior. No se reescribió el historial de `git` —hacerlo sobre ramas ya compartidas con el equipo habría roto sus copias locales—; la aclaración se hace por escrito aquí y en `CONTRIBUTORS.md`, que es el mecanismo apropiado para una corrección de autoría a posteriori.

Adicionalmente, varios *commits* de Ricardo y Darwin usan correos personales en vez del institucional, y la cuenta de Ricardo aparece con dos identidades (`ricardoforever@outlook.es` y `ricardo@email.com`). Las cifras de la tabla las consolidan como una sola persona. Se recomienda unificar `git config user.email` antes de la Entrega Final.

17. Trazabilidad y honestidad académica

La matriz `docs/trazabilidad/matriz.csv` enlaza cada requisito con su historia de usuario, su caso de uso, su *endpoint*, el archivo que lo implementa, la prueba JUnit que lo cubre y la evidencia empírica correspondiente.

El historial de `git` evidencia la autoría de cada integrante. Se documenta en `CONTRIBUTORS.md` que dos *commits* figuran bajo una cuenta distinta por haber sido realizados desde un equipo de la universidad con otra sesión configurada; la aclaración se hace por escrito en lugar de reescribir

el historial ya compartido. El mismo archivo declara explícitamente el uso de herramientas de asistencia por inteligencia artificial, siguiendo la guía de transparencia del SWEBOK v4.0 [35].

18. Discusión

Nota de versión: esta sección responde a las preguntas de investigación (sección 1.2) con la evidencia empírica disponible al momento de escribirla. Los bloques marcados [ACTUALIZAR] dependen de una corrida fresca de `make bench` / `make test` sobre el estado actual de `main` — la evidencia archivada en `docs/mediciones/` antecede al módulo de inventario y a la mitad del dominio deportivo, así que reportar esos números tal cual sin remedirlos sería exactamente el error que la sección 12.3 ya documentó una vez (un indicador verde que no refleja el sistema real).

RQ1 — Rendimiento. La corrida de referencia disponible midió $p95 = 14,18$ ms contra el *endpoint* de listado protegido, muy por debajo del umbral de 200 ms con caché caliente. [ACTUALIZAR] esa cifra corresponde a 3 corridas, no a las 5 independientes que exige la Entrega Final, y no queda registrado si distingue explícitamente el escenario de caché fría (umbral 500 ms) del escenario de caché caliente. Falta también el test de Wilcoxon con tamaño de efecto (Cliff's delta) entre ambos escenarios que pide el Bloque C. Con 5 corridas nuevas por escenario, RQ1 debería poder responderse con el mismo margen holgado que ya se observó, pero eso es una expectativa, no todavía un resultado.

RQ2 — Seguridad. Los seis controles OWASP (A01, A02, A03, A05, A07, A09) están verificados con evidencia reproducible en `docs/mediciones/sec/` (sección 12.2.1), y una revisión directa del código fuente y de `db/procs/` no encontró ningún uso de `createNativeQuery` con concatenación, `@Query` construida con `+`, ni `EXECUTE/sp_executesql` dentro de los procedimientos almacenados — la estrategia híbrida no introdujo la superficie de ataque que introduciría SQL dinámico mal construido. [ACTUALIZAR] falta el escaneo automático con OWASP ZAP en modo *baseline* y el análisis estático (find-sec-bugs, regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE`) que la Entrega Final exige como evidencia adicional, reproducible por un tercero sin depender de esta revisión manual.

RQ3 — Usabilidad. La respuesta es clara y es, a la vez, el hallazgo menos cómodo de todo el documento (sección 12.7): la usabilidad **no** es homogénea entre roles. El sistema obtiene grado A en entrenadores y estudiantes y grado F en recepcionistas y representantes, con una media agregada (68,25) que esconde ese patrón bimodal en vez de exponerlo. La implicación práctica es directa: el esfuerzo de mejora de interfaz debería priorizar el flujo administrativo y el futuro módulo de representantes, no el promedio general del sistema. [ACTUALIZAR] con $N = 10$ participantes la desagregación por perfil es indicativa, no concluyente; subir a $N \geq 15$ (mínimo que exige la Entrega Final) permitiría un intervalo de confianza más ajustado por subgrupo.

RQ4 — Cobertura y trazabilidad bajo crecimiento. Esta pregunta nace directamente de un patrón que ya se repitió una vez en la Segunda Entrega (sección 6): la cobertura cayó a 39,8 % cuando se agregaron cinco recursos nuevos sin pruebas propias, y se corrigió a 72,5 %. La pregunta para la Entrega Final es si ese patrón se repite con el módulo de inventario y el resto del dominio deportivo, agregados después de esa medición. [ACTUALIZAR] el reporte archivado en `docs/mediciones/jacoco/` es anterior a esos módulos — no los incluye — y `pom.xml` hoy exige un mínimo de 60 % medido solo sobre `LINE`, con `deportivo/**` y varios paquetes de `seguridad/academico` excluidos explícitamente del chequeo. Antes de poder responder RQ4 con evidencia hace falta una corrida de `make test` sobre el `main` actual, sin exclusiones, con `BRANCH` incluido en la regla. La respuesta provisional, mientras tanto, es una advertencia: el patrón de

la Segunda Entrega es exactamente el tipo de regresión silenciosa que este proyecto ya demostró que puede ocurrir dos veces si no se vuelve a medir explícitamente.

19. Trabajo futuro

Cinco líneas de trabajo quedan fuera del alcance de esta entrega, cada una con una razón concreta documentada en otra parte del proyecto — no son una lista genérica de ideas, son los pendientes que `docs/etica/ETHICS.md` y `VERSIONING.md` ya declaran abiertos.

1. **Envío proactivo de notificaciones al representante (RF-22).** La lectura de informes por un representante con vínculo activo ya funciona; el envío proactivo (push/email/SMS) queda fuera a propósito hasta no tener un mecanismo de consentimiento explícito y fechado, distinto del consentimiento general de matrícula (hallazgo H-04, resuelto parcialmente el 2026-08-03).
2. **Mecanismo de supresión de datos.** El sistema aplica baja lógica, no borrado físico, por integridad referencial del historial deportivo. Si un representante ejerciera el derecho de supresión, hoy no hay un procedimiento que lo satisfaga. Se propone un procedimiento almacenado de anonimización — sustituir datos identificativos por valores neutros conservando claves foráneas y estadísticas agregadas, invocable solo por **ADMINISTRADOR** y registrado en auditoría (hallazgo H-03).
3. **Base legal documentada para peso y altura.** Estos dos campos son datos de salud de un menor sin finalidad de uso verificada, en el esquema pero sin ningún *endpoint* de escritura confirmado (hallazgo H-06). La línea de trabajo es doble: documentar una finalidad concreta con consentimiento separado, o directamente no exponerlos por API si ninguna funcionalidad prevista los necesita.
4. **Validación y protección de la cédula.** Hoy es un `VARCHAR(10)` sin validación de dígito verificador, sin restricción de unicidad y sin cifrado en reposo (hallazgo H-01) — más permisivo de lo deseable para un identificador nacional de un menor en un despliegue con datos reales.
5. **Cerrar la brecha de usabilidad por rol.** La sección anterior (RQ3) encontró grado F en recepcionistas y representantes. Rediseñar específicamente esos dos flujos —no el sistema en general— y volver a medir SUS por subgrupo con $N \geq 15$ es la línea de trabajo con el respaldo empírico más directo de todo este documento.

20. Conclusiones

1. El sistema cumple con holgura los umbrales cuantitativos exigidos: p95 de 14,18 ms frente a un límite de 200 ms, 6 de 6 controles OWASP verificados con evidencia reproducible, accesibilidad Lighthouse de 100/100, y **72,5 %** de cobertura de pruebas frente a un mínimo de 60 % (sección 12.5) — cifra alcanzada después de detectar y corregir una regresión real a 39,8 % causada por los recursos nuevos de la reestructuración, no reportando directamente un número cómodo.
2. **Una medición que no se ejecuta contra el sistema real no es evidencia.** Es la lección transversal de esta entrega, y aparece tres veces con la misma forma. La cobertura de JaCoCo se midió sobre un `target/` con clases previas a la reestructuración e incluía paquetes que ya no existen. La auditoría OWASP A01 devolvía 401 en todo y eso se parecía a un resultado correcto, cuando en realidad su propio control A07 la había dejado bloqueada. Y el registro de usuarios (RF-01) estaba roto contra la base de datos desde la reestructuración mientras sus dos pruebas unitarias pasaban, porque mockean el repositorio y la restricción la aplica PostgreSQL. En los tres casos el indicador era verde y el sistema no hacía lo que el indicador

decía. Las tres mediciones se corrigieron en su origen — `make test` pasa a `clean test`, la auditoría limpia su propio contador y comprueba recurso por recurso— para que el defecto no pueda repetirse sin ser visible.

3. El aporte más sustantivo de esta entrega no fue añadir funcionalidad, sino *verificar* lo que ya existía, dos veces: una vez contra el código de la entrega original, y una segunda vez después de que otros dos integrantes reestructuraran el backend en paralelo. Ambas rondas revelaron defectos que ninguna revisión superficial había detectado — desde que JaCoCo no medía cobertura en absoluto, hasta que un ADR de arquitectura describía un mecanismo de autenticación (JWT en `localStorage`) distinto del que el código realmente usa (cookie `HttpOnly`), pasando por el control de acceso ausente en los cinco recursos nuevos (H-08). Todos los defectos encontrados se corrigieron y quedan documentados en las secciones 6 y 12.3 en vez de ocultarse.
4. La distinción entre lo implementado y lo modelado se mantiene explícita en todo el documento. En el caso de **Representante** y **Equipo** se llevó un paso más allá: en vez de dejar catorce clases vacías —una de ellas de cero bytes, el mismo defecto que motivó tres observaciones del docente en la Entrega 1B— se eliminaron del código y ambos módulos quedan declarados como pendientes solo en la documentación (sección 6). Un archivo vacío no cuenta como funcionalidad entregada, y tampoco debería quedarse en el repositorio aparentando que sí.
5. La encuesta SUS (sección 12.7) ya no es un frente abierto, pero su resultado abre uno nuevo: el sistema es excelente para entrenadores y estudiantes (grado A) e inaceptable para receptionistas y padres de familia (grado F). Mejorar la usabilidad del flujo administrativo y del futuro módulo de representantes es, con evidencia real detrás, más urgente que subir la media agregada. La exposición REST del dominio deportivo restante (horarios, sesiones, asistencias, evaluaciones) sigue siendo el objetivo natural de la Entrega Final.

21. Declaraciones obligatorias

Sección propia y consolidada, exigida por el Bloque B.15 de la Guía de la Entrega Final. Las declaraciones de ética y de contribuciones (CRediT) ya se desarrollaron con su propio detalle en las secciones correspondientes de este documento; aquí se referencian en vez de duplicarse, junto con las declaraciones que todavía no tenían una sección propia.

Disponibilidad de código. Repositorio público en https://github.com/DarwinSM21/SGED_APPWEB, etiqueta `v1.0.0` **pendiente** (ver nota de versión en la portada), DOI del software en Zenodo: [10.5281/zenodo.21713240](https://doi.org/10.5281/zenodo.21713240).

Disponibilidad de datos. Los datos crudos de todas las mediciones (rendimiento, seguridad, cobertura, calidad web, usabilidad) están versionados en `docs/mediciones/` bajo la misma licencia del repositorio. **Pendiente:** el depósito separado del *dataset* de mediciones en Zenodo con DOI propio y licencia Creative Commons Attribution 4.0, distinto del DOI del software, que exige el Bloque D.3.

Disponibilidad de materiales. Colección Postman (`docs/postman/coleccion.json`, 32 peticiones), *scripts* de carga k6 (`k6/`), configuración Lighthouse (`lighthouse.js`), instrumento y protocolo SUS (`docs/mediciones/sus/INSTRUMENTO-SUS.md`), plantilla de consentimiento (`docs/etica/consentimiento/`). **Pendiente:** los análisis de rendimiento y de SUS son *scripts* de Python (`scripts/perf-analysis.py`, `scripts/sus-analysis.py`) y no cuadernos ejecutables con salidas archivadas (Jupyter o RMarkdown) como exige literalmente el Bloque D.1; convertirlos o documentar la equivalencia es una tarea abierta antes del cierre.

Declaración ética. El sistema trata datos personales de menores de edad. Los participantes de la encuesta SUS firmaron consentimiento informado, sus respuestas se anonimizaron con códigos P01–P10, y ocho hallazgos éticos (H-01 a H-08, uno ya corregido) están declarados sin omisiones en `docs/etica/ETHICS.md` y en la sección de Consideraciones éticas de este documento.

Contribuciones de autores (CRediT). Los catorce roles de la taxonomía CRediT asignados a cada integrante, derivados del historial de `git` y no autodeclarados, se detallan en la sección 16 y en `CONTRIBUTORS.md`.

Conflictos de interés. El equipo declara la ausencia de conflictos de interés: ningún integrante tiene relación contractual, financiera o personal con la escuela ProFútbol ni con ningún proveedor mencionado en este documento (Fly.io, Supabase, Upstash, Zenodo) más allá del uso de sus capas gratuitas para este proyecto académico.

Financiamiento. Este trabajo no recibió financiamiento externo. Es un Proyecto Fin de Curso de la asignatura Aplicaciones Web, desarrollado como requisito académico de la Universidad Técnica Estatal de Quevedo, sin patrocinio de terceros.

Uso de asistentes de inteligencia artificial. Siguiendo la guía de transparencia del SWE-BOK v4.0 [35], el equipo declara el uso de un asistente de IA generativa (Claude, de Anthropic) como herramienta de apoyo en revisión y corrección de código de seguridad, generación de *scripts* de auditoría y análisis de resultados, y redacción de documentación (incluidas partes de este informe). El diseño de la arquitectura, las decisiones de seguridad y la verificación de que el sistema funciona correctamente fueron hechos y revisados por el equipo. El detalle completo, con el alcance exacto por archivo, está en `CONTRIBUTORS.md`.

Agradecimientos. Al Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D., por la dirección y retroalimentación del PFC a lo largo de las cuatro entregas, y a la Universidad Técnica Estatal de Quevedo por el acceso a la infraestructura académica del curso.

Referencias

- [1] Asamblea Nacional del Ecuador. Ley orgánica de protección de datos personales. Registro Oficial Suplemento 459, 26 de mayo de 2021, 2021.
- [2] Association for Computing Machinery. ACM code of ethics and professional conduct. <https://www.acm.org/code-of-ethics>, 2018.
- [3] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: A critical review and guidelines. *Empirical Software Engineering*, 27(4):94, 2022.
- [4] Aaron Bangor, Philip Kortum, and James Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.
- [5] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In *Encyclopedia of Software Engineering*, volume 1, pages 528–532. John Wiley & Sons, 1994.
- [6] Tobias Blobel and Martin Lames. A concept for club information systems (cis) — an example for applied sports informatics. *International Journal of Computer Science in Sport*, 19(1), 2020.
- [7] John Brooke. SUS: A ‘quick and dirty’ usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [8] Simon Brown. *Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture*. Leanpub, 2018. Modelo C4.
- [9] Diana Castilla, Irene Jaén, Carlos Suso-Ribera, Gemma García-Soriano, Irene Zaragoza, Juana Bretón-López, Adriana Mira, Amanda Díaz-García, and Azucena García-Palacios. Psychometric properties of the spanish full and short forms of the system usability scale (SUS): Detecting the effect of negatively worded items. *International Journal of Human-Computer Interaction*, 40(15), 2023.
- [10] Tse-Hsun Chen, Weiyi Shang, Jinqiu Yang, Ahmed E. Hassan, Michael W. Godfrey, Mohamed Nasser, and Parminder Flora. An empirical study on the practice of maintaining object-relational mapping code in Java systems. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, pages 165–176, 2016.
- [11] Alistair Cockburn. *Writing Effective Use Cases*. Agile Software Development Series. Addison-Wesley Professional, Boston, MA, 2001.
- [12] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, MA, 2004.
- [13] Emelie Engström, Margaret-Anne Storey, Per Runeson, Martin Höst, and Maria Teresa Baldassarre. How software engineering research aligns with design science: A review. *Empirical Software Engineering*, 25(4):2630–2660, 2020.
- [14] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [15] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [16] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.

- [17] Muhammad Imran, Vittorio Cortellessa, Davide Di Ruscio, Riccardo Rubei, and Luca Traini. Is code coverage of performance tests related to source code features? an empirical study on open-source Java systems. *Empirical Software Engineering*, 30(6):157, 2025.
- [18] International Council on Systems Engineering. INCOSE guide to writing requirements. Technical Report INCOSE-TP-2010-006-04, International Council on Systems Engineering (INCOSE) — Requirements Working Group, July 2023.
- [19] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.
- [20] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering, 2018.
- [21] M. B. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). Request for Comments 7519, Internet Engineering Task Force (IETF), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [22] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-001, Keele University and Durham University Joint Report, July 2007.
- [23] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [24] Yixuan Liu. Design and implementation of a student attendance management system based on springboot and vue technology. *Frontiers in Computing and Intelligent Systems*, 8(1):91–97, 2024.
- [25] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs. Request for Comments 9457, Internet Engineering Task Force (IETF), July 2023. <https://www.rfc-editor.org/rfc/rfc9457>.
- [26] Michael T. Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011.
- [27] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
- [28] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [29] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Heidelberg, 2010.
- [30] PostgreSQL Global Development Group. PostgreSQL 16 documentation — CREATE PROCEDURE. <https://www.postgresql.org/docs/16/sql-createprocedure.html>, 2026.
- [31] Janandani Ranaweera, Dan Weaving, Marcelo Zanin, Matthew C. Pickard, and Gregory Roe. Digitally optimizing the information flows necessary to manage professional athletes: A case study in rugby union. *Frontiers in Sports and Active Living*, 4:850885, 2022.
- [32] Redgate. Flyway documentation — migrations. <https://documentation.red-gate.com/flyway/>, 2026.
- [33] O. Suria. A statistical analysis of system usability scale (SUS) evaluations in online learning platform. *Journal of Information Systems and Informatics*, 6(2), 2024.

- [34] Saara Tenhunen, Tomi Männistö, Matti Luukkainen, and Petri Ihantola. A systematic literature review of capstone courses in software engineering. *Information and Software Technology*, 159:107191, 2023.
- [35] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [36] Shao-Fang Wen and Basel Katt. A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. *Computers & Security*, 135:103532, 2023.
- [37] Jingcheng Yang, Enze Wang, Jianjun Chen, Qi Wang, Yuheng Zhang, Haixin Duan, Wei Xie, and Baosheng Wang. Token time bomb: Evaluating JWT implementations for vulnerability discovery. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.